VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018



A PROJECTREPORT

On

"The Campus Event Aggregator (Digital Noticeboard)"

A Dissertation Submitted in partial fulfillment of the requirement for the degree of

BACHELOR OF ENGINEERING IN COMPUTER SCIENCE AND ENGINEERING

Submitted by

KOUSHIK M(1RG23CS025)

NIKILESWARN(1RG23CS044)

NAMITHA M C (1RG23CS041)

MANYA M (1RG23CS032)

Under The Guidance of

Ms.MEGHASHREE V

Assistant Professor, Dept. of CSE

RGIT, Bengaluru-32



Department of Computer Science & Engineering

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

Cholanagar, R. T. Nagar Post, Bengaluru-560032

2026-2027

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated to Visvesvaraya Technological University)

Cholanagar, R.T. Nagar Post, Bengaluru-560032

Department of Computer Science & Engineering

CERTIFICATE

Phase-II

This is to certify that the Project Report entitled "The Campus Event Aggregator (Digital Noticeboard)" is a Bonafide work carried out by **Mr. KOUSHIK M (1RG23CS025), Mr. NIKILESWAR N (1RG23CS044), Ms. N A M I T H A M C (1RG23CS041), Ms. MANYAM (1RG23CS032)** in partial fulfillment for the award of **Bachelor of Engineering** in **Computer Science and Engineering** under **Visvesvaraya Technological University, Belagavi,** during the year **2026- 2027.** It is certified that all corrections/suggestions given for Internal Assessment have been incorporated in the report. This Project report has been approved as it satisfies the academic requirements in respect of project work (BCS786) prescribed for the said degree.

Signature of guide	Signature of HOD	Signature of Principal
Ms. Meghashree V	**Dr. Arudra A**	**Dr. D G Anand**
Assistant Professor	Associate Professor, HOD	Principal, RGIT,
Dept. of CSE	Dept. of CSE	Benagluru-32
RGIT, Bengaluru-32	RGIT, Bengaluru-32	

External Viva

Name of the Examiners	**Signature with date**
1.	
2.	

VISVESVARAYATECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

DEPARTMENTOF COMPUTER SCIENCE & ENGINEERING

DECLARATION

We hereby declare that the project work entitled **" The Campus Event Aggregator(Digital Noticeboard)"** submitted to the **Visvesvaraya Technological University, Belagavi** during the academic year **2026-2027**, is record of an original work done by us under the guidance of **Mrs. Soniya Komal V, Mrs. Bhagyashree Wakde** Assistant professor, Rajiv Gandhi Institute of Technology , Bengaluru and this project work is submitted in the partial fulfillment of requirements for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering.** The results embodied in this project have not been submitted to any other University or Institute for award of any degree or diploma.

KOUSHIK M (1RG23CS025)

MANYA M (1RG23CS032)

NAMITHA M C (1RG23CS041)

NIKILESWAR N (1RG23CS044)

ACKNOWLEDGEMENT

We take this opportunity to thank our college, **Rajiv Gandhi Institute of Technology, Bengaluru**, for providing us with an opportunity to carry out this project work. We express our gratitude to **Dr. D G Anand**, Principal, RGIT, Bengaluru, for providing the resources and support without which the completion of this project would have been a difficult task.

We extend our sincere thanks to **Dr. Arudra A**, Associate Professor and Head, Department of Computer Science & Engineering, RGIT, Bengaluru, for his constant support and encouragement, and for being a pillar of strength in the face of all adversities.

We would like to acknowledge the guidance and support extended towards us by our project coordinators, **Mrs. Soniya Komal V** and **Mrs. Bhagyashree Wakde**, Assistant Professors, Department of CSE, RGIT, Bengaluru. Their encouragement and valuable technical support gave us the environment to enhance our knowledge and skills and to reach the pinnacle with sheer determination, dedication and hard work. We are especially grateful to our mentor, **Ms. Meghashree V**, Assistant Professor, Department of CSE, RGIT, Bengaluru, for her unwavering guidance and support, whose expertise was instrumental in the success of this project.

We also want to extend our thanks to the entire faculty and support staff of the Department of Computer Science & Engineering, RGIT, Bengaluru, who have encouraged us throughout the course of the Bachelor's Degree. We want to thank our family for always being there with full support and for providing us with a safe haven to conduct and complete our project. Lastly, we are grateful to all our friends for the helpful insights they gave us during this project.

KOUSHIK M	**(1RG23CS025)**
MANYA M	**(1RG23CS032)**
NAMITHA M C	**(1RG23CS041)**
NIKILESWAR N	**(1RG23CS044)**

ABSTRACT

University campuses are home to countless events throughout the academic year. Hackathons, cultural competitions, technical workshops, and sports tournaments all compete for student attention. Yet despite the energy that clubs and committees pour into organizing these events, many students simply never hear about them. The reason is not a lack of interest — it is a broken communication system.Today, event information is scattered across WhatsApp groups, Instagram stories, and physical posters that get torn down overnight. There is no single platform a student can visit to see everything happening on campus. Opportunities are missed. Registrations close before students even knew they were open.

The Campus Event Aggregator — "The Digital Noticeboard" — is a centralized web-based platform that solves this problem. Student clubs can log in as Admins and post, update, or remove events. Students can browse all events, search by category, view a calendar, and register directly through the platform. Built on the MERN stack (MongoDB, Express.js, React.js, Node.js), the project demonstrates the three core pillars of modern web development: Authentication, CRUD Operations, and Database Management.

CONTENTS

ACKNOWLEDGEMENT		**i**
ABSTRACT		**ii**
LIST OF FIGURES		**vi**
LIST OF TABLES		**vii**
LIST OF ABBREVIATIONS		**ix**

CHAPTER NO	TITLE	PAGE NO
1	**INTRODUCTION**	**1**
1.1	**Background**	**1**
1.2	**Problem Statement**	**2**
1.3	**Objectives**	**2**
1.4	**Contributions**	**3**
1.5	**Scope of the Project**	**4**
1.6	**Motivation**	**4**
1.7	**Methodology Followed**	**5**
1.8	**Organization of the Report**	**5**
2	**LITERATURE SURVEY**	**6**
2.1	**Survey of Existing Systems**	**6**
2.2	**Personalized Event Recommendation**	**8**
2.3	**Smart Campus and Student Event Platforms**	**8**
2.4	**Intelligent Notifications and Personalized Scheduling**	**9**
2.5	**Research Gap**	**9**
3	**SYSTEM ANALYSIS AND REQUIREMENTS**	**10**
3.1	**Requirements Gathering**	**10**
3.2	**Functional Requirements**	**10**
3.3	**Non-Functional Requirements**	**11**
3.4	**Actors and Use Cases**	**11**
3.5	**Hardware and Software Requirements**	**13**
3.6	**Feasibility Study**	**13**
3.7	**Constraints, Assumptions, and Dependencies**	**14**
3.8	**Risk Analysis**	**14**
3.9	**User Stories**	**14**
3.10	**Data Requirements**	**15**
4	**SYSTEM DESIGN**	**16**
4.1	**Description of Modules**	**16**
4.2	**Module Interaction**	**18**
4.3	**System Overview**	**18**
4.4	**System Architecture**	**19**
4.5	**User Roles**	**20**
4.6	**System Workflow**	**20**

4.7	Data Flow Design	21
4.8	Database Design	22
4.9	API Design	25
4.10	AI-Based Event Recommendation	26
4.11	Smart Notification	27
4.12	Security Design	28
4.13	Design Decisions and Alternatives	28
5	**IMPLEMENTATION**	**30**
5.1	Technical Stack	30
5.2	Frontend Technologies	30
5.3	Backend Technologies	31
5.4	Database Management System	32
5.5	API and Integration Technologies	32
5.6	Authentication and Security Technologies	32
5.7	AI and Recommendation Technologies	33
5.8	Cloud and Hosting Platforms	33
5.9	Comparison of Alternative Technologies	33
5.10	Software Quality Practices	34
5.11	Project Structure	34
5.12	Frontend	34
5.13	Backend	36
5.14	Database	38
5.15	AI Module	40
5.16	Notification Module	40
5.17	Calendar Module	41
5.18	Deployment and Configuration	41
6	**TESTING AND RESULTS**	**43**
6.1	Testing Strategy	43
6.2	Unit Test Cases	43
6.3	Integration Test Cases	44
6.4	System Test Cases	44
6.5	Results and Discussion	44
A	**SCREENSHOTS**	**44**
A.1	User Interface Wireframes	44
B	**SAMPLE CODE / PSEUDOCODE**	**46**
B.1	Algorithms	46
B.2	Additional Backend Components	46
C	**ADDITIONAL RESULTS**	**52**
C.1	Security Test Cases	52
C.2	Automated Test Implementation	52
C.3	Performance Evaluation Plan	52
C.4	Traceability of Objectives	53
D	**SUPPORTING DOCUMENTS**	**54**
D.1	Project Phases	54
D.2	Milestones	54
D.3	Prerequisites	55
D.4	Configuring the Server	55
	CONCLUSION AND FUTURE SCOPE	**56**

LIST OF FIGURES

Fig. 3.1: Use case diagram of the Campus Event Aggregator 12
Fig. 4.1: System architecture of the Campus Event Aggregator 19
Fig. 4.2: Deployment view of the system 20
Fig. 4.3: System workflow of the Campus Event Aggregator 21
Fig. 4.4: Level 0 data flow diagram (context diagram) 21
Fig. 4.5: Level 1 data flow diagram 22
Fig. 4.6: Entity–relationship diagram 23
Fig. 4.7: Processing pipeline of the recommendation module 26
Fig. A.1: Wireframes of the student dashboard (illustrative) 44
Fig. A.2: Wireframe of the organizer console (illustrative) 44
Fig. A.3: Wireframes of the personal calendar and the admin approval queue (illustrative) 45
Fig. D.1: Gantt Chart 54

LIST OF TABLES

Table 2.1: Literature Survey 7
Table 2.2: Capability comparison of existing and proposed systems 8
Table 2.3: Research gaps and the corresponding design response 9
Table 3.1: Functional requirements 11
Table 3.2: Non-functional requirements 11
Table 3.3: Use case specification for event registration 12
Table 3.4: Development environment 13
Table 3.5: Deployment environment 13
Table 3.6: Risk analysis 14
Table 3.7: User stories and the requirements that satisfy them 15
Table 4.1: Dependencies between modules 18
Table 4.2: Role-based permissions 20
Table 4.3: Database Entities 22
Table 4.4: User collection 23
Table 4.5: Event collection 24
Table 4.6: Registration collection 25
Table 4.7: Attendance log collection 25
Table 4.8: Major APIs 25
Table 4.9: Supporting API endpoints 25
Table 4.10: Events used in the worked example 27
Table 4.11: Notification Parameters 27
Table 4.12: Threat analysis using STRIDE 28
Table 4.13: Design decisions and alternatives 29
Table 5.1: Technical Stack 30
Table 5.2: Comparison of technology alternatives 33
Table 6.1: Levels of testing 43
Table 6.2: Unit test cases 43
Table 6.3: Integration test cases 44
Table 6.4: System test cases 44
Table 6.5: Comparison of the earlier and extended versions 45
Table C.1: Security test cases 52
Table C.2: Performance targets 53
Table C.3: Traceability between objectives and tests 53
Table D.1: Project phases 54
Table D.2: Meaning of configuration variables 55

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CSE	Computer Science and Engineering
DFD	Data Flow Diagram
ER	Entity–Relationship
HTTP / HTTPS	Hypertext Transfer Protocol / Secure
iCalendar	Standard format (.ics) for calendar data
IDF	Inverse Document Frequency
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
ML	Machine Learning
NFR / FR	Non-Functional Requirement / Functional Requirement
ODM	Object Document Mapper
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SMS	Short Message Service
TF	Term Frequency
TF-IDF	Term Frequency–Inverse Document Frequency
TLS	Transport Layer Security
UI / UX	User Interface / User Experience
WSS	WebSocket Secure

CHAPTER 1

INTRODUCTION

1.1 Background

A university campus is more than classrooms and examinations. It is a living community where students develop leadership, creativity, and professional skills through extracurricular participation. Student clubs organize events ranging from coding contests to cultural performances, but the way these events are communicated has not kept pace with students' digital habits. Most institutions still depend on WhatsApp broadcasts, printed posters, and word of mouth. A message buried under hundreds of unread chats, or a poster soaked by rain, serves no purpose.

First-year students are hit the hardest. New to campus, unfamiliar with the social network, and not yet added to every relevant group, they are the most likely to miss events that could shape their college experience. The Campus Event Aggregator was first developed to bring event discovery and registration onto a single platform. This report presents an extended version of that platform, which adds AI-based event recommendation, smart notifications, and a personalized event calendar for students.

1.1.1 The Role of Extracurricular Events in Campus Life

Technical and cultural events are a core part of the learning experience in higher education. Hackathons, workshops, seminars, guest lectures, and cultural festivals let students apply classroom knowledge, build a peer network, and discover career directions. Employers, admission committees, and accreditation bodies also look at participation in such activities as evidence of holistic development.

A typical engineering campus hosts dozens of events every semester, and many overlap in the same week while competing for the same student body. Without a dependable way of publicising and organising them, organizers' effort is not fully rewarded and students benefit far less than they could.

1.1.2 Limitations of Current Communication Channels

Existing practice relies on four informal channels, each with structural weaknesses.

- **Messaging groups.** WhatsApp and Telegram groups are fast, but messages scroll away within hours, are mixed with casual chat, and reach only some students.
- **Printed posters and notice boards.** Posters reach only students passing a corridor, are easily covered, and cannot be updated when plans change.
- **Email circulars.** Official emails are credible but bulk, rarely personalised, and often ignored.
- **Word of mouth.** Informal sharing favours the socially connected and leaves newcomers uninformed.

None of these channels provides registration, attendance tracking, or analytics, so organizers cannot tell how many students are interested, how many attended, or which events draw the largest audience.

1.1.3 Need for an Intelligent Event Platform

A single platform that aggregates all events solves the discovery problem, but not the relevance problem: as the catalogue grows, students face a long list of mostly irrelevant events. A platform that learns from a student's interests and past participation and surfaces the most relevant events first is far more likely to be used regularly. Likewise, a platform that notifies for every event will be muted, whereas one sending a few timely, relevant alerts will be trusted. These two needs, personalization and selective notification, motivate the extended Campus Event Aggregator presented here.

1.2 Problem Statement

The core problem is the absence of a single, trustworthy source of campus event information. Students are forced to monitor multiple disconnected channels simultaneously, and even then there is no guarantee they will receive timely or accurate updates. Existing event management systems also lack personalization, provide only basic alerts and simple attendance records, and remain vulnerable to misuse.

This fragmentation causes three issues. The first is **information inequality**: only students in the right group or near the right notice board are informed. The second is a **lack of credibility**: anyone can post in a group chat, so official announcements are hard to tell from casual messages. The third affects clubs: without structured registration, organizers cannot measure interest, plan logistics, or keep records. An intelligent college event management system is therefore needed, one that recommends events by user preferences, sends timely notifications, and provides a personal event calendar.

1.2.1 Formal Problem Definition

Let S be the set of students on a campus, E the set of events published during a semester, and C the set of clubs that organize them. Each student s in S has an interest profile and a history of registrations and attendances. Each event e in E has a title, description, category, venue, time window, and an organizing club. The problem is to design a system that, for every student s, (a) presents the events in E ordered by their relevance to s, and (b) notifies s about a bounded number of high-value events at appropriate times.

1.2.2 Consequences of the Existing Situation

The consequences of the current situation can be observed from three perspectives. From the student's perspective, valuable opportunities are missed and engagement drops, particularly among first-year students. From the organizer's perspective, events are under-attended despite good planning, and there is no reliable data with which to evaluate success or to justify budgets. From the institution's perspective, the participation records needed for accreditation, reward schemes, and student-development reporting are incomplete and difficult to verify.

1.3 Objectives

The objective of this project is to design and develop a functional, role-based web application that serves as a unified platform for campus event management and discovery. The system is built around two user roles. **Club Administrators / Organizers** are verified users who represent student clubs or organizing committees; they can create

new event listings, update existing ones, delete cancelled events, and track attendance. **General Students** can register on the platform, browse all active events, search and filter by category or date, view a calendar of upcoming events, receive recommendations and notifications, and register for events.

The primary objectives of the proposed system are:

- To bring all information about college events onto a single platform for convenient access and registration.
- To recommend events to users based on their interests, profile, and past attendance.
- To send smart notifications that reduce missed events and irrelevant alerts.
- To offer a personal event calendar for managing registered and recommendedevents.
- To increase student engagement through a convenient and personalized event experience.

Beyond its practical purpose, the project is also a deliberate learning exercise covering authentication through secure, role-based login; CRUD operations through full event lifecycle management; and database management through structured storage of users, events, registrations, notifications, and attendance in MongoDB. The frontend is built with React.js and Tailwind CSS, and the backend is powered by Node.js and Express.js, with Socket.IO for real-time updates.

1.3.1 Measurable Goals

To make the objectives verifiable, the following measurable goals were set for the system.

- A student must be able to register for an event in no more than three interactions from the event catalogue.
- The recommendation list must be produced for a student in under one second for a catalogue of several hundred events.
- The same account must not be able to hold more than one registration for the same event.
- Every secured endpoint must reject requests that carry no valid token or that come from a role without permission.

1.4 Contributions

- **AI-Based Event Recommendations:** recommending suitable events based on the interests, profile, and participation history of students.
- **Smart Notifications:** delivering relevant event notifications while avoiding missed events and irrelevant alerts.
- **Personalized Event Calendar:** organizing registered and recommended events for each student.
- **Campus Platform Integration:** combining event search, registration, recommendation, notification, attendance, and management in a single platform.
- **Higher Student Engagement:** a more convenient and personalized way to engage in campus activities.

1.5 Scope of the Project

This project covers the complete development of a web application intended for deployment within a single university campus.

What is included in this version:

- User registration and role-based login (Admin/Organizer and Student)
- Full CRUD functionality for event management by Club Admins
- Event browsing, searching, and filtering for Students
- Calendar view displaying registered and recommended events
- Student registration for individual events
- Content-based AIevent recommendation (TF-IDF and cosine similarity)
- Smart in-app and email notifications
- Real-time organizer dashboard with attendance analytics
- Responsive design for both desktop and mobile browsers

What is outside the current scope:

- Payment processing for ticketed events
- A dedicated native mobile application
- Real-time messaging between students and clubs
- Multi-campus or multi-institution deployment
- SMS delivery of notifications and full production rollout of the AI service (planned as future work)

These excluded features represent logical extensions of the platform and are earmarked for future development phases.

1.5.1 Intended Users and Stakeholders

The primary users are students and club organizers. Secondary stakeholders include faculty coordinators who approve events, the administration office that needs participation records, and placement or training cells that may use participation data to understand student interests. The system is designed for a single institution with a few thousand students and several dozen active clubs, which is the size at which the problems described above are most acute and where a modest cloud deployment is sufficient.

1.6 Motivation

Every project begins with a reason. For this one, the reason was frustration. Imagine finding out about a hackathon after it already happened. Good prizes. Great networking. Exactly what you needed. You missed it — not because you were busy, but because nobody told you. This situation happens to students every single day on campuses across the country. Students are surrounded by opportunities — coding contests, cultural competitions, guest lectures, and

entrepreneurship challenges — yet most of the student body remains unaware of them. The events are well-organized. The communication system is not.

The earlier version of the system left room for improvement in personalization, security, and usability, so this work adds AI-based recommendations, notifications, and a personalized calendar. There is also a social motivation: awareness of campus events is often a social privilege, and a centralized public platform removes that inequality.

The motivation was also technical. Building role-based authentication, live database management, a recommendation engine, and a responsive calendar interface forces a developer to grow. Finally, this project is a portfolio piece: a working full-stack application proves practical engineering ability — the ability to see a problem and build a solution from scratch.

1.7 Methodology Followed

The project was carried out using an incremental development approach. The first increment delivered the core platform: authentication, event management, search, and registration. The second increment, which is the subject of this report, added the recommendation service, notification rules, the personal calendar, and the organizer analytics dashboard. Each increment was planned, designed, implemented, tested, and reviewed before the next one started. Requirements were gathered through informal discussions with students and club organizers, through observation of how events were advertised and attended on campus, and through a review of existing event management systems described in Chapter 2.

1.8 Organization of the Report

Chapter 2 surveys existing event management systems and related research, and identifies the research gap. Chapter 3 presents system analysis and requirements, including use cases, feasibility, and risks. Chapter 4 describes the system design: modules, architecture, data model, API design, and the recommendation and notification components. Chapter 5 explains the technologies and implementation. Chapter 6 documents testing and results. Chapter 7 concludes the report and outlines future scope. The references and appendices follow.

CHAPTER 2

LITERATURE SURVEY

2.1 Survey of Existing Systems

Many web-based event management systems have been reported for colleges and universities. Table 2.1 summarises ten representative works with their methodology, contributions, and remaining gaps. Most automate scheduling, registration, and communication; very few address personalization.

S. No	Author(s) & Year	Title of Paper	Methodology/ Approach	KeyFeatures / Contributions	Limitations / Research Gap
1	Chaturvedi et al. (2024)	CU-Events: A Comprehensive Event Management System for University	Three-layerarchitecture (Frontend, Backend, Database)	Centralized platform, real-time tracking, improved student engagement,automated workflows	Lacks advanced recommendation systems and personalization features
2	Smithet al. (2020)	Online Event Management System for Educational Institutions	Web-basedautomation system	Event scheduling, notifications, registrationautomation, improved communication	Limitedanalytics and personalization capabilities
3	Kumar& Sharma (2021)	AutomatedCollege EventManagement Using Web Technologies	Database-drivenweb application	Real-time updates, SMS/email notifications, improved data accuracy	Does not address scalability and predictiveinsights
4	Mishra et al. (2023)	EventManagement System	Web-basedapplication with user/admin modules	Booking system, centralized database, user registration, vendor interaction	Focuses on general events, not campus-specific aggregation
5	Uikeyet al. (2025)	Web-Based Campus Event Management System	Integrated web system with real-time tools	Event scheduling, registration,reminders, data analytics for decision-making	Limitedfocus on recommendation engines and personalization
6	Bawankar et al. (2023)	EventManagement System	Online system replacing manual processes	Efficientevent handling, cost and time saving, centralized access	Does not incorporate intelligent features like AI-based suggestions
7	IJSRET Survey (2025)	College Event Management:Analytics and Personalization	Analyticaland case-study approach	Focuson improving engagement, communication, personalization and analytics	Early-stage research in analytics and personalization
8	IJARCCE (2025)	University Event ManagementUsing Data Visualization	Data-driven system using visualization tools	Insights into student interests, trend analysis, centralized platform	Requires large datasets and lacks real-time recommendation optimization

S. No	Author(s) & Year	Title of Paper	Methodology/ Approach	KeyFeatures / Contributions	Limitations / Research Gap
9	IRJET (2020)	College Event Management System	Traditionalweb-based system	Manages college fests, cultural and technical events efficiently	Limited scalability and lacks modern UX/UI enhancement
10	IJARSCT Survey (2025)	Evolving Trends in College Event Management	Survey-based study	Highlightsdigital transformation, automation, and engagement improvements	Identifies need for intelligentaggregation and personalization

Table 2.1: Literature Survey

2.1.1 Discussion of the Surveyed Systems

Centralised university platforms. Chaturvedi et al. describe CU-Events, a three-layer web application that places event listing, registration, and tracking in one place. It automates earlier manual workflows and uses the same three-layer pattern adopted here, but it shows every student the same unfiltered event list and does not adapt to the user.

Automation-oriented systems. Smith et al. and Kumar and Sharma automate scheduling, registration, and notifications with database-driven pages and SMS or e-mail alerts. Their notifications are broadcast to all users, causing alert fatigue, and scalability and predictive insight remain uncovered.

General event management systems. Mishra et al. and Bawankar et al. describe general-purpose systems with user and administrator modules and booking. Because they target commercial events as much as campus events, they include irrelevant features such as vendor interaction and omit campus needs such as club identity and department filtering.

Analytics-oriented work. Uikey et al., IJSRET, and IJARCCE move towards dashboards, trend analysis, and visualization of student interests. Their analysis is mostly retrospective: insight reaches administrators after the fact rather than reaching students as personalised suggestions while events are still open.

Traditional systems and surveys. The IRJET system manages festivals efficiently but has a dated interface and limited scalability, while the IJARSCT survey concludes that intelligent aggregation and personalization are the next step, which supports the direction taken here.

2.1.2 Feature Comparison

Table 2.2 compares the capabilities of the surveyed systems with those of the proposed system; “Partial” means the capability is present in a limited form.

Capability	Basic web systems (items 2, 3, 6, 9)	Centralizedplatforms (items 1, 4, 5)	Analytics-oriented(items 7, 8, 10)	Proposed system
Central event listing	Yes	Yes	Yes	Yes
Online registration	Yes	Yes	Partial	Yes
Search and filtering	Partial	Yes	Partial	Yes
Personalised recommendation	No	No	Partial	Yes

Capability	Basic web systems (items 2, 3, 6, 9)	Centralizedplatforms (items 1, 4, 5)	Analytics-oriented(items 7, 8, 10)	Proposed system
Smart(selective) notification	No	Partial	No	Yes
Personalevent calendar	No	Partial	No	Yes
Real-timeorganizer dashboard	No	Partial	Partial	Yes
Role-basedaccess control	Partial	Yes	Partial	Yes

Table 2.2: Capability comparison of existing and proposed systems

2.2 Personalized Event Recommendation

Event recommendation has been studied extensively in event-based social networks: SoCaST (Ogundele et al.) models geographical, categorical, social, and temporal preferences, EventRec uses user and event features, SERGE (Liu et al.) models successive events, and Bok et al. analyzed participation histories. Most such work targets general social networks rather than colleges.

2.2.1 Approaches to Recommendation

Recommender systems fall into three families. **Content-based filtering** recommends items similar to those a user liked before and works for new items immediately. **Collaborative filtering** recommends items that similar users liked but needs large interaction data and suffers from cold start. **Hybrid approaches** combine both and are common in large commercial systems.

Events add a further difficulty: they are short-lived, so an event may be over before enough users register to feed collaborative filtering, and every semester brings first-year students with no history. This work therefore adopts a content-based approach first and leaves collaborative signals for future work.

2.2.2 Text Representation with TF-IDF

TF-IDF represents a text document as a numerical vector: a term's weight grows with its frequency in the document and shrinks with the number of documents containing it, so distinctive words weigh more than common ones. Combined with cosine similarity, which compares the angle between vectors regardless of length, it gives a simple, fast, and interpretable way of ranking events against a student's interests.

2.3 Smart Campus and Student Event Platforms

Smart-campus research uses digital technologies to improve campus services. Yang et al. presented a situational-awareness framework, Zhang et al. discussed technologies for human-centered smart campuses, and Dong et al. examined frameworks for integrated smart-campus services. The earlier Campus Event Aggregator supported discovery, rewards, dashboards, and real-time communication but lacked personalized event discovery, leaving room for AI-driven recommendation.

2.3.1 Relevance to Event Management

Smart-campus literature treats the campus as a connected environment where sensing, communication, and data analysis improve safety, learning, administration, and student life. Event management is one of its most visible student-facing services, since it involves time-sensitive information and physical presence. A platform handling discovery, registration, and notification is therefore both a useful service and a source of data on campus usage.

2.4 Intelligent Notifications and Personalized Scheduling

Research on intelligent notifications analyzes user context, behavior, and reaction models to decide the timing and relevance of alerts, and uses machine learning to predict user reactions. These results support personalized notification strategies on campus, whereas traditional event systems send alerts unrelated to a student's interests or schedule.

2.4.1 Notification Fatigue

Studies of mobile and web notifications report that too many low-value alerts lead users to disable notifications altogether. Good design therefore needs relevance (the alert concerns something the user cares about), timeliness (it arrives while the user can still act), and restraint (information is not repeated). These properties are reflected in the four parameters of this project's notification module: user interest, event timing, registration status, and notification history.

2.5 Research Gap

The reviewed literature treats event recommendation, smart-campus applications, and intelligent notifications largely as independent areas, and little attention has been paid to an integrated platform combining them for the college community. The proposed system therefore extends the earlier Campus Event Aggregator with intelligent notifications and AI-based personalized event scheduling.

2.5.1 Summary of Gaps Addressed

The gaps identified in this survey, and the way the proposed system responds to each of them, are listed in Table 2.3.

Gap identified in the literature	Response in the proposed system
Recommendation research targets general event-based social networks, not colleges	Content-based recommender using campus event text, categories, and student interests
Cold-start problem for new students and new events	TF-IDF profile built from declared interests, so recommendations work from the first login
Notifications are broadcast to everyone	Rule-based filtering using interest, timing, registration status, and history
Analytics are retrospective and only for administrators	Live organizer dashboard using Socket.IO and aggregation pipelines
Features are studied as isolated systems	One integrated MERN platform combining all capabilities

CHAPTER 3

SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Requirements Gathering

Requirements were collected through informal interviews with students from all four years, conversations with coordinators of technical and cultural clubs, a review of the systems surveyed in Chapter 2, and observation of how events were advertised and attended during one semester. Three themes emerged: students learned about events too late or not at all; organizers did not know how many people would come and could not stop one person signing the attendance sheet for another; and faculty coordinators needed reliable participation records. These themes became the requirements below.

3.2 Functional Requirements

The functional requirements describe what the system must do. They carry FR codes so they can be traced to design elements and to the test cases in Chapter 6.

ID	Requirement	Role
FR-01	The system shall allow a new user to register with name, e-mail, password, department, and interests.	Student
FR-02	The system shall authenticate users and issue a signed token that expires after a fixed period.	All
FR-03	The system shall allow an organizer to create an event with title, description, category, venue, date, time, and capacity.	Organizer
FR-04	The system shall allow an organizer to edit or delete only the events belonging to the organizer's own club.	Organizer
FR-05	The system shall display all published, upcoming events in a single catalogue.	Student
FR-06	The system shall allow searching by keyword and filtering by category, date, venue, and club.	Student
FR-07	The system shall allow a student to register for an event and shall prevent a second registration for the same event.	Student
FR-08	The system shall refuse registration when the event is full, closed, or cancelled.	Student
FR-09	The system shall confirm every successful registration to the student.	System
FR-10	The system shall recommend events to a student, ranked by similarity to the student's interests and history.	Student
FR-11	The system shall send in-app and e-mail notifications according to the notification rules.	System
FR-12	The system shall display a personal calendar with registered and recommended events.	Student
FR-13	The system shall allow an organizer to mark attendance for a registered student.	Organizer
FR-14	The system shall reject a duplicate attendance entry for the same registration with a clear reason.	System
FR-15	The system shall show attendance on the organizer dashboard in real time.	Organizer

ID	Requirement	Role
FR-16	The system shall allow an administrator to approve events, verify clubs, and manage users.	Admin
FR-17	The system shall allow a student to submit a rating and comment for an event they attended.	Student
FR-18	The system shall provide analytics on registrations, attendance, and categories.	Organizer, Admin

Table 3.1: Functional requirements

3.3 Non-Functional Requirements

Non-functional requirements describe the qualities the system must have. They are identified by codes beginning with NFR.

ID	Category	Requirement
NFR-01	Performance	Catalogue and search responses should be returned within one second for several hundred events.
NFR-02	Performance	A registration request should be validated and acknowledged within two seconds.
NFR-03	Scalability	The system should support a few thousand students with a burst of several hundred registrations within a short interval at the start of a popular event.
NFR-04	Security	Passwords shall be stored only as salted hashes; all traffic shall use HTTPS; secured routes shall require a valid token and the correct role.
NFR-05	Reliability	The database shall be replicated so that a single node failure does not interrupt service; the target availability is at least 99.5 per cent.
NFR-06	Usability	The student interface shall be usable on a phone screen without horizontal scrolling and shall require no training.
NFR-07	Maintainability	The code shall be modular, with business logic separate from the HTTP layer, so that each module can be tested independently.
NFR-08	Portability	The application shall run on current versions of Chrome, Firefox, Edge, and Safari on desktop and mobile.
NFR-09	Privacy	Only data needed for the service shall be collected; interests and attendance shall be visible only to the student and to authorised organizers.
NFR-10	Extensibility	The notification transport and the recommendation service shall be replaceable without changes to the calling code.

Table 3.2: Non-functional requirements

3.4 Actors and Use Cases

Three actors interact with the system: the **Student**, the **Club Administrator / Organizer**, and the **System Administrator**. Fig. 3.1 shows the use cases of each actor; registration and login are shared by all three.

3.4.1 Use Case Specifications

The most important use case, event registration, is specified in detail in Table 3.3.

Field	Description
Use case	UC-03: Register for an event
Actor	Student

Field	Description
Precondition	The student is logged in and the event is published and open for registration
Main flow	1. The student opens the event page. 2. The student selects Register. 3. The system checks capacity and duplicate registration. 4. The system creates the registration. 5. The system queues a confirmation e-mail. 6. The event is added to the student's calendar.
Alternate flow	The event is full or closed: the system shows a message and no registration is created. The student has already registered: the system shows the existing registration.
Postcondition	A registration exists and a notification has been created

Table 3.3: Use case specification for event registration

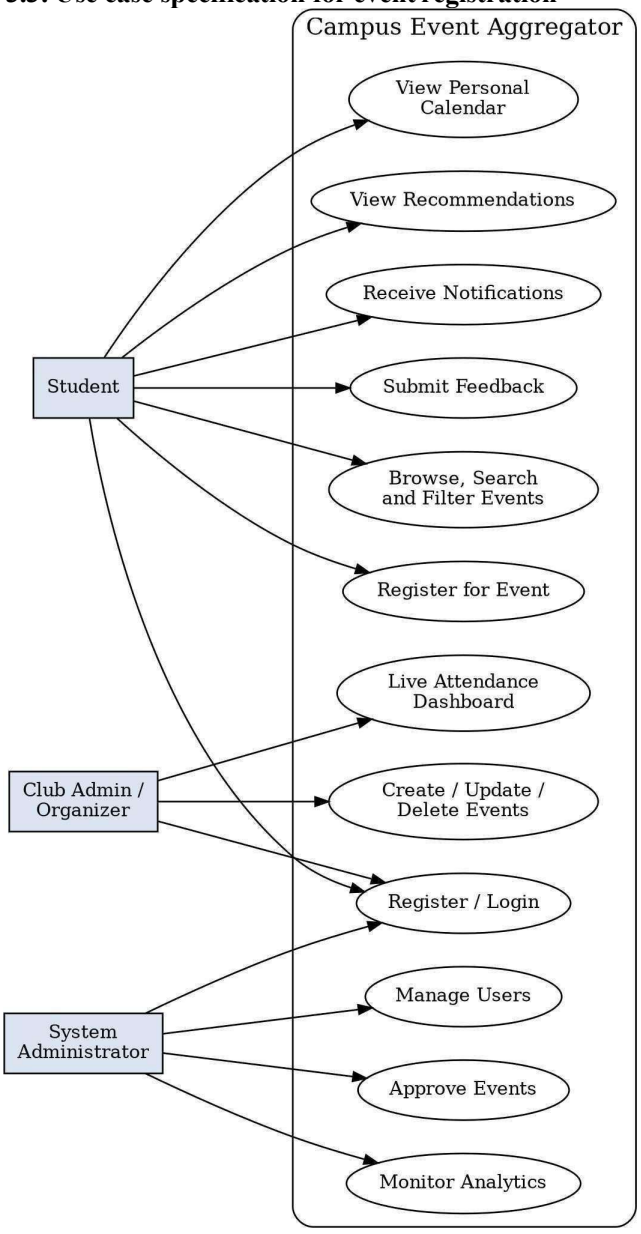


Fig. 3.1: Use case diagram of the Campus Event Aggregator

3.5 Hardware and Software Requirements

3.5.1 Development Environment

Item	Specification
Processor	Dual-core 2.0 GHz or better
Memory	8 GB RAM recommended
Storage	10 GB free disk space
Operating system	Windows 10/11, Ubuntu 22.04 or later, or macOS
Runtime	Node.js 18 LTS or later, npm 9 or later
Database	MongoDB 6 or later (local instance or managed cluster)
Tools	Git, Visual Studio Code, Postman, MongoDB Compass
Browser	Latest Chrome, Firefox, Edge, or Safari

Table 3.4: Development environment

3.5.2 Deployment Environment

Item	Specification
Application server	2 vCPU and 4 GB RAM virtual machine or container service
Database	Managed MongoDB replica set with automated backups
Static hosting	CDN or web server for the React build
Network	HTTPS with a valid certificate; WebSocket support enabled
Mail	Authenticated SMTP account with sufficient daily quota
Client devices	Any phone, tablet, or computer with a current browser

Table 3.5: Deployment environment

3.6 Feasibility Study

3.6.1 Technical Feasibility

All of the technologies used are open-source, mature, and well documented. JavaScript is used throughout the stack, so one developer can work on the frontend, backend, and recommendation service. E-mail delivery and real-time communication libraries are readily available through npm, and the TF-IDF and cosine similarity computations are light enough for an ordinary server. The project is therefore technically feasible.

3.6.2 Economic Feasibility

The software stack carries no licence cost, and a laptop plus free tiers of managed database and hosting services suffice for development and a pilot. A full campus deployment needs only a small virtual machine, a managed database, a domain name, and an e-mail service, which are modest compared with printing posters and maintaining attendance sheets. The project is therefore economically feasible.

3.6.3 Operational Feasibility

The interface follows patterns that students already know from social and shopping applications, and the organizer console is limited to a few clear screens. The system is therefore operationally feasible.

3.6.4 Schedule Feasibility

The work was divided into increments of two to three weeks each (timeline in Appendix D). Because the earlier version already provided authentication and event management, the second increment could focus on the new modules and finish within the academic term.

3.7 Constraints, Assumptions, and Dependencies

- **Single campus.** The system is designed for one institution; multi-campus tenancy is out of scope.
- **Connectivity.** Students and organizers are assumed to have a working data or Wi-Fi connection at the venue.
- **E-mail reliability.** E-mail was unreliable during the pilot, so the in-app channel is primary and e-mail a supplement.
- **Interest data.** Recommendation quality depends on meaningful interests; attendance history is also used when available.
- **Data protection.** Personal data is stored only for the purposes described and is handled in line with institutional policy.

3.8 Risk Analysis

Risk	Likelihood	Impact	Mitigation
Unreliable e-mail delivery	High	Low	In-app notification as primary channel; pluggable transport
Cold start for new students	High	Medium	Interest-based profile vector and content-based ranking
Misuse of organizer privileges	Low	High	Club verification, role-based access, audit trail

Table 3.6: Risk analysis

3.9 User Stories

User stories express the requirements from the user's point of view and were used to discuss priorities with students and organizers. Each story is linked to the functional requirement that implements it.

ID	As a...	I want to...	So that...	Requirement
US-01	New student	Choose my interests when I sign up	I see relevant events on my first day	FR-01, FR-10
US-02	Student	See all campus events in one place	I do not have to check many groups	FR-05
US-03	Student	Search and filter by category and date	I can find what suits my schedule	FR-06
US-04	Student	Register in a few taps	I do not queue or fill paper forms	FR-07, FR-09
US-05	Student	Be warned when two events overlap	I can avoid a clash	FR-12
US-06	Student	Receive only useful reminders	I am not overwhelmed by messages	FR-11

ID	As a...	I want to...	So that...	Requirement
US-07	Organizer	Publish an event once for the whole campus	Students from every department can see it	FR-03
US-09	Organizer	See attendance as it happens	I can plan seating, food, and certificates	FR-15
US-11	Administrator	Approve events and verify clubs	Only genuine events appear	FR-16
US-12	Administrator	See participation statistics	I can report on student engagement	FR-18

Table 3.7: User stories and the requirements that satisfy them

3.10 Data Requirements

The system stores personal and activity data, and the requirements below define what is kept and for how long.

- **Personal data.** Name, college e-mail, department, year, and password hash. No phone number, address, or government identifier is collected.
- **Preference data.** Interests selected by the student, used only for recommendation and notification and editable at any time.
- **Activity data.** Registrations, attendance logs, notifications, feedback, and reward points, retained for the academic year to support certificates and reports.
- **Operational data.** Audit entries for administrative actions and manual attendance, retained for the same period.
- **Access.** Students see all of their own data; organizers see only the registrations and attendance of their own events; administrators see aggregated data and the audit trail.
- **Removal.** On request, a student account can be deleted; registrations are anonymised so that event statistics remain correct.

CHAPTER 4

SYSTEM DESIGN

4.1 Description of Modules

4.1.1 User Management Module

Handles registration, login, and profile management for student and organizer/administrator roles.

- Inputs: name, e-mail, password, department, year of study, and interests chosen from the event categories.
- Processing: validates input, checks the e-mail is unused, hashes the password, assigns the student role, and stores the profile; organizer status needs administrator verification.
- Outputs: a user record, a signed access token on login, and a profile used by the recommendation and notification modules.

4.1.2 Event Creation and Management Module

Lets organizers and administrators create, update, and delete events.

- **Inputs:** title, description, category, venue, start and end time, capacity, and an optional poster.
- Processing: validates that the end time follows the start time, confirms the organizer owns the event's club, and stores the record; changes or cancellations alert registered students via the notification module.
- **Outputs:** an event record in the *draft*, *pending*, *published*, or *cancelled* state.

4.1.3 Event Aggregation Module

Collects events from all departments and clubs into one centralized listing.

- **Inputs:** published events from every club on campus.
- **Processing:** merges them into one catalogue, sorts by date, and hides events that have ended or been cancelled.
- Outputs: a paginated, uniform list of active events used by the search and calendar modules.

4.1.4 Search and Filter Module

Lets users find events by keyword, category, date, or location, with sorting and filtering.

- **Inputs:** a keyword, category, date range, venue, or club, plus a sort order.
- Processing: builds a MongoDB query using a text index on title and description plus indexed category and date filters.
- **Outputs:** a filtered and sorted page of events returned in a few milliseconds.

4.1.5 Registration Module

Allows users to register for events online, manages participant data, and confirms registrations.

- **Inputs:** the authenticated student and the identifier of the event.

- Processing: checks that the event is open and not full, rejects duplicate registrations through a compound unique index, and stores the registration.
- **Outputs:** a registration record and a confirmation notification.

4.1.6 AI Recommendation Module

Suggests events from users' interests, profile, and past activity by ranking cosine similarity of TF-IDF vectors, which also works for new students.

- Inputs: the student's interests, registration and attendance history, and upcoming events.
- Processing: preprocesses text, builds TF-IDF vectors for events and the user, computes cosine similarity, removes registered events, and sorts the rest.
- **Outputs:** a ranked list of the top N events with a similarity score for each.

4.1.7 Smart Notification Module

Keeps users updated about relevant events, changes, and reminders while avoiding irrelevant or repeated alerts.

- Inputs: triggers such as a new event, an event change, or an approaching start time.
- Processing: applies the notification rules, suppresses duplicates, and chooses the delivery channel.
- Outputs: in-app notifications pushed via Socket.IO and e-mails queued for the mail worker.

4.1.8 Calendar and Scheduling Module

Provides a personal calendar combining registered and AI-recommended events, helping users avoid conflicts.

- **Inputs:** the student's registrations and the current recommendation list.
- Processing: places events on month, week, or agenda views, colours registered versus recommended events, and flags overlapping slots.
- **Outputs:** a calendar view and an optional downloadable iCalendar (.ics) file.

4.1.9 Admin Dashboard Module

Lets administrators manage users, approve events, monitor activity, and view live attendance analytics.

- **Inputs:** administrator actions such as approving an event, verifying a club, or suspending an account.
- **Processing:** performs the action after checking the administrator role and records it in an audit trail.
- **Outputs:** updated records and a summary of platform activity.

4.1.10 Analytics and Reporting Module

Analyzes participation and event performance with MongoDB aggregation pipelines and charts.

- **Inputs:** registrations, attendance logs, and feedback for a club or for the whole platform.
- **Processing:** groups and counts records by event, category, department, and time using aggregation pipelines.

- Outputs: registrations versus attendance, attendance by hour, and category popularity, shown as Chart.js graphs.

4.1.11 Feedback and Review Module

Collects reviews and ratings after events so organizers can understand satisfaction.

- Inputs: a rating from one to five and an optional comment, accepted only from registered students.
- **Processing:** stores one review per student per event and recalculates the average rating of the event.
- **Outputs:** review lists and rating summaries for organizers.

4.1.12 Security and Authentication Module

Implements JWT login, role-based access control, input validation, secure headers, and rate limiting.

- **Inputs:** every incoming HTTP request and socket connection.
- Processing: verifies the token and role, validates the body, applies rate limiting, and sets protective headers.
- Outputs: a request passed on to the controller or rejected with a suitable status code.

4.1.13 Integration and User Interface Module

Connects the system with email and calendar services through a responsive, mobile-first interface.

- **Inputs:** requests from the React application and responses from the API.
- Processing: renders role-specific views, manages Redux state, validates forms, and connects to e-mail and calendar export.
- **Outputs:** the screens used by students, organizers, and administrators.

4.2 Module Interaction

The modules interact: Table 4.1 shows what each module depends on and receives from others, which also indicates their implementation order.

Module	Depends on	Data received
Registration	User Management, Event Management	Authenticated student, open event
AI Recommendation	User Management, Event Aggregation, Registration	Interests, event text, history
Smart Notification	Event Management, Registration, Recommendation	Triggers, interest match, registration status
Calendar	Registration, AI Recommendation	Registered and recommended events
Analytics	Registration, Feedback	Counts, timestamps, ratings
Admin Dashboard	User Management, Event Management, Analytics	Pending events, user list, summaries
Feedback	Registration	Registered participation

Table 4.1: Dependencies between modules

4.3 System Overview

The proposed Campus Event Aggregator is a central platform where students discover, register for, and manage

college events, with personalized recommendations, notifications, and a personal calendar.

The design follows three principles. The first is **single source of truth**: every event is published once, by a verified organizer. The second is **relevance over volume**: the platform ranks and filters rather than broadcasting. The third is **modularity**: each capability sits behind a small interface and can change independently.

4.4 System Architecture

The system has a user interface, an application server, a database, and AI-enabled services (Fig. 4.1). The server manages registration, notifications, and recommendations, the database stores users, events, registrations, and attendance, and the AI services generate recommendations and notifications.

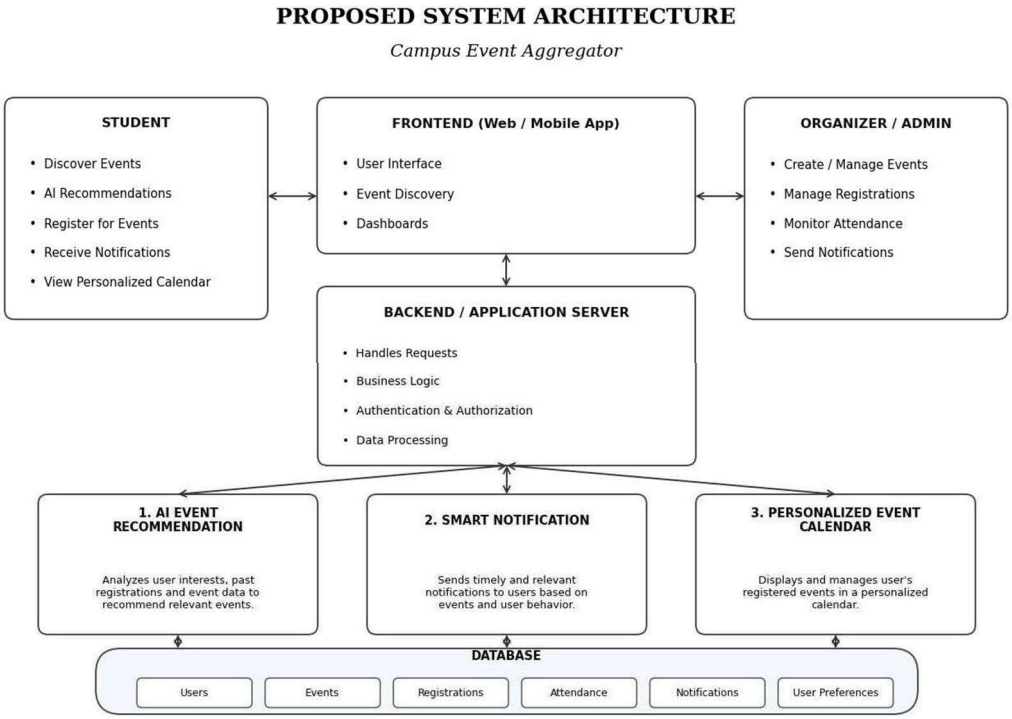


Fig. 4.1: System architecture of the Campus Event Aggregator

4.4.1 Presentation Layer

The presentation layer is a single-page React application with Student, Organizer, and Admin views, using REST calls, a WebSocket for live updates, and a Redux store; it is mobile-first.

4.4.2 Application Layer

The application layer is a Node.js and Express server. Requests pass through a fixed middleware chain (security headers, rate limiting, token check, role check, validation) before reaching a controller that calls testable service functions; Socket.IO pushes live updates.

4.4.3 Data Layer

The data layer is a replicated MongoDB deployment accessed through Mongoose, with schemas for field types, required fields, allowed values, and references. Indexes support the two hot paths: catalogue search and student–event uniqueness.

4.4.4 Intelligence Layer

The intelligence layer holds the recommendation service and the notification rules engine. The recommendation service is a separate Node.js process returning ranked lists over a small REST interface, so it can be scaled, replaced, or switched off independently.

4.4.5 Deployment View

Fig. 4.2 shows the deployment. Browsers connect through a reverse proxy that terminates HTTPS and forwards requests and WebSocket traffic; the React build is served statically; and the API, recommendation service, and mail worker run as separate processes sharing the database.

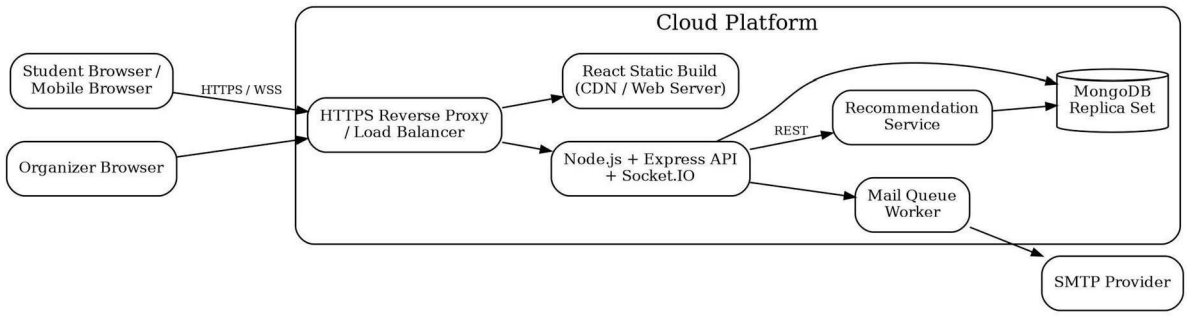


Fig. 4.2: Deployment view of the system

4.5 User Roles

The solution defines two roles: Student and Organizer/Administrator. Students discover events, receive recommendations and notifications, register, and use a personal calendar; administrators create and manage events, handle registrations, and track attendance.

Capability	Student	Organizer	Administrator
Register and log in	Yes	Yes	Yes
Browse, search, and filter events	Yes	Yes	Yes
Register for events	Yes	No	No
View recommendations and personal calendar	Yes	No	No
Create, edit, and delete own club's events	No	Yes	Yes
View live attendance	No	Yes	Yes
View analytics for own events	No	Yes	Yes
Approve events and verify clubs	No	No	Yes
Manage users and view platform analytics	No	No	Yes

Table 4.2: Role-based permissions

4.6 System Workflow

As shown in Fig. 4.3, a student signs in, browses events, and receives recommendations based on interests and past actions. After registering for an event, the student gets timely notifications and the event is added to the personal calendar.

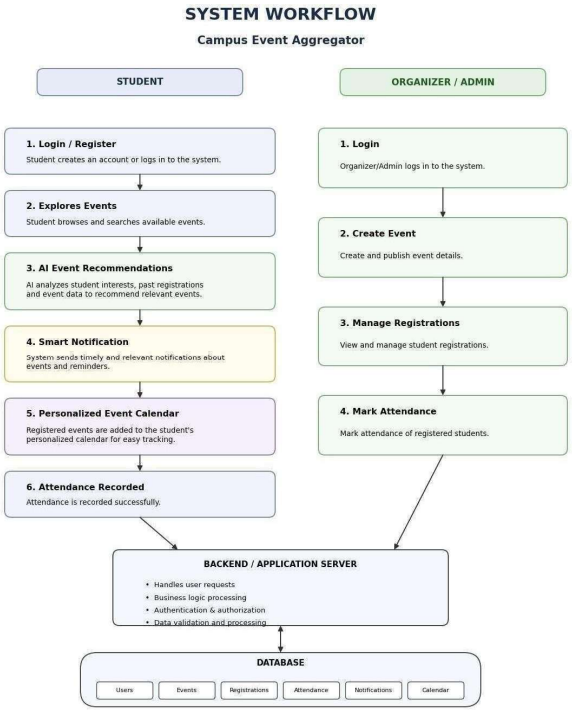


Fig. 4.3: System workflow of the Campus Event Aggregator

4.6.1 Event Lifecycle

An event passes through a few states. An organizer creates it as a draft; on submission it is pending until an administrator or faculty coordinator approves and publishes it. After the end time it is completed and attendance and feedback are frozen. Cancellation before the end time notifies every registered student.

4.7 Data Flow Design

The context diagram (Fig. 4.4) treats the platform as one process interacting with students, organizers, administrators, and the e-mail server; Fig. 4.5 decomposes it into six processes and five data stores.

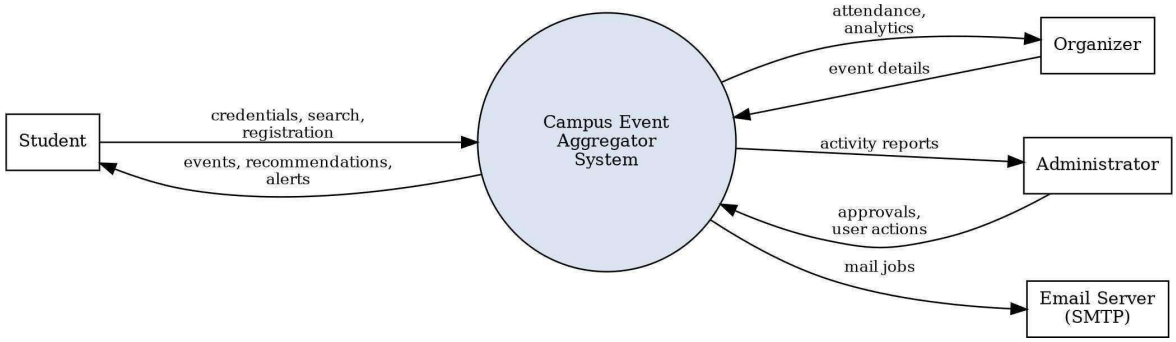


Fig. 4.4: Level 0 data flow diagram (context diagram)

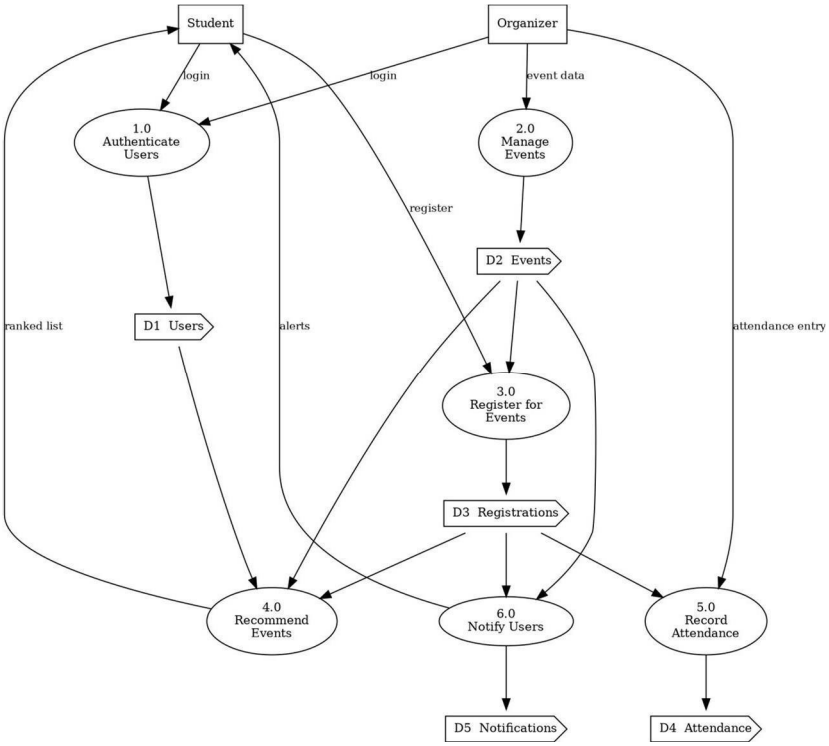


Fig. 4.5: Level 1 data flow diagram

4.8 Database Design

The database stores user, event, registration, notification, and attendance information, summarized in Table 4.3; the AI recommendation module also consumes user interests and event details.

Entity	Purpose
User	Stores account details: name, email, password hash, role, department, regId, semester
Event	Stores event details, registered students, and embedded team registrations
Attendance	Stores attendance records (student, event, time)
Timetable	Stores class timetable entries per department and semester

Table 4.3: Database Entities

Fig. 4.6 shows the entity relationships: a club organizes many events; a student makes many registrations; an event has many registrations; each registration has at most one attendance log; and students receive many notifications and earn many rewards.

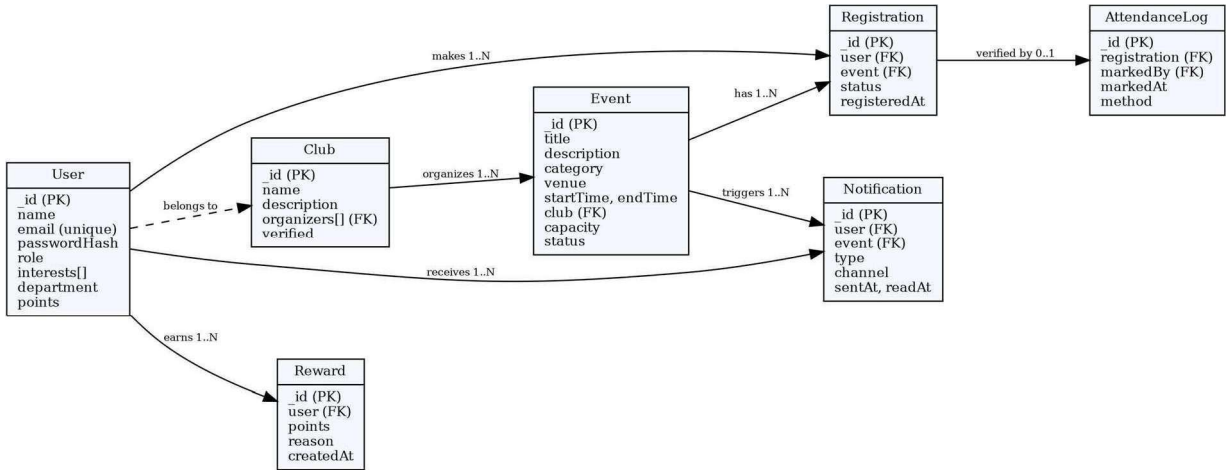


Fig. 4.6: Entity–relationship diagram

4.8.1 Collection Schemas

The principal fields of the four most important collections are listed below.

Field	Type	Constraint / Description
_id	ObjectId	Primary key
name	String	Required
email	String	Required, unique
password	String	Required; stored as a bcrypt hash
role	String	One of student, faculty, admin; default student
department	String	Optional; used to match the timetable
regId	String	Required; college registration number
semester	Number	1 to 8; required only when role is student

Table 4.4: User collection

Field	Type	Constraint / Description
_id	ObjectId	Primary key
title	String	Required, trimmed
description	String	Required
category	String	Required
department	String	Required
date	Date	Required
venue	String	Required
organizer	ObjectId	Reference to User who created the event; required
registrationDeadline	Date	Optional
maxParticipants	Number	Required; maximum students in one team
registeredStudents	[ObjectId]	References to User; team leaders who registered
teamRegistrations	[Object]	Embedded team registrations (Table 4.6)
points	Number	Default 0; reward points for the event

Field	Type	Constraint / Description
_id	ObjectId	Auto-generated id of the embedded team registration
teamLeader	ObjectId	Reference to the User who registered the team
teamSize	Number	Required; at most maxParticipants of the event
teamMembers	[String]	Names of the other members; length equals teamSize minus 1

Table 4.6: Team registration sub-document (Event.teamRegistrations)

Field	Type	Constraint / Description
_id	ObjectId	Primary key
student, event	ObjectId	References to User and Event; both required
checkedInAt	Date	Default: current time
status	String	present or invalid; default present
createdAt, updatedAt	Date	Added by Mongoose timestamps

Table 4.7: Attendance collection

4.9 API Design

REST APIs connect the frontend and backend (Table 4.8). All endpoints other than registration and login require a valid JWT access token. In the current version the token identifies the user, but the routes do not yet restrict access by role.

API	Method	Purpose
/api/auth/login	POST	User authentication
/api/auth/register	POST	Create a user account
/api/events	GET / POST	View and create events
/api/events/:id	PUT / DELETE	Update / delete an event
/api/events/:id/register	POST	Team registration for an event
/api/attendance/mark	POST	Mark attendance

Table 4.8: Major APIs

4.9.1 Extended API Reference

Endpoint	Method	Role	Purpose
/api/auth/profile	GET	Any user	Return the decoded token payload
/api/auth/students/semester/:semester	GET	Any user	List students of a semester
/api/events/:id	GET	Any user	Event with organizer and registered students
/api/attendance/event/:eventId	GET	Any user	Attendance records of an event
/api/timetable	POST / GET	Any user	Add / list timetable by department and semester
/api/timetable/:id	DELETE	Any user	Delete a timetable entry
/	GET	Public	Server status message

Table 4.9: Supporting API endpoints

4.10 AI-Based Event Recommendation

The recommendation module aligns student interests with event information. It builds a TF-IDF matrix from event titles, descriptions, and categories and computes the cosine similarity between a user vector U and an event vector E:

$$Similarity(U, E) = (U \cdot E) / (|U| \, |E|) \quad (4)$$

Events with a higher similarity are ranked higher in the recommendation list.

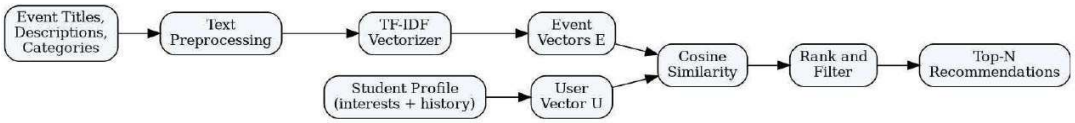


Fig. 4.7: Processing pipeline of the recommendation module

4.10.1 Text Preprocessing

Text is converted to lower case, punctuation and numerals are removed, stop words are discarded, and each word is stemmed. The category label is repeated twice so that it carries more weight than an ordinary word. The same steps apply to the student's interests and to events the student has registered for or attended.

4.10.2 TF-IDF Weighting

For a term t in a document d from a collection of N documents, the weight is the product of the term frequency and the inverse document frequency. Here tf(t, d) is the number of times t appears in d and df(t) is the number of documents containing t. Each vector is then normalised to unit length so that long descriptions do not dominate.

$$w(t, d) = tf(t, d) \times idf(t), \quad idf(t) = \ln((1 + N) / (1 + df(t))) + 1 \quad (5)$$

4.10.3 Building the User Vector

Declared interests are treated as a short document and give the base of the vector, which suffices for a new student. For a returning student, vectors of registered or attended events are added; attended events weigh more than registered-only ones, and recent events more than old ones. The combined vector is normalised before similarity is computed.

4.10.4 Worked Example

Take three events and a student with interests coding, web, and react. The preprocessed terms are in Table 4.10. The term web appears in two of three events, so idf = ln(4/3) + 1 = 1.288; coding and react appear in one event each, so idf = ln(4/2) + 1 = 1.693. After normalisation the cosine similarity is 0.459 for E3, 0.416 for E1, and 0.000 for E2. The web workshop is ranked first, the hackathon second, and the cultural night, which shares no term with the interests, is not recommended.

Event	Terms after preprocessing
E1: Hackathon	hackathon, coding, contest, build, web, apps
E2: Cultural night	classical, dance, cultural, night
E3: Web workshop	web, development, workshop, react, node

Table 4.10: Events used in the worked example

4.10.5 Filtering and Ranking

The module removes ended events, events already registered, and events below a small similarity threshold. The rest are sorted by score, ties broken by the nearer start date, and the top N are returned, with a cap per category so that the list is not one type of event.

4.11 Smart Notification

The smart notification module informs students about relevant events while reducing noise, using the parameters in Table 4.11.

Parameter	Purpose
User interest	Checks event relevance
Event timing	Provides timely reminders
Registration	Prioritizes registered events
Notification history	Avoids repeated alerts

Table 4.11: Notification Parameters

4.11.1 Notification Rules

- **New event.** A notification is created only for students whose similarity score for the event exceeds the notification threshold, and for students in the organizing club.
- **Registered event reminders.** Students who have registered receive a reminder one day before the event and another one hour before it.
- **Event change or cancellation.** Every registered student is notified immediately, regardless of other rules, because the information affects their plans.
- **Duplicate suppression.** A notification of the same type for the same event is never sent to the same student twice.
- **Daily cap.** A student receives at most a fixed number of discretionary notifications per day; mandatory notifications such as cancellations are not counted.
- **Confirmation.** Successful registration generates one confirmation.

4.11.2 Channel Selection

In-app notifications go over the open WebSocket, or are stored for the next login if the student is offline. E-mail (confirmations and reminders) is queued and sent by a worker, so a slow mail provider cannot delay a registration response.

4.12 Security Design

Security is implemented in layers, so that a failure in one layer does not expose the system.

- **Authentication.** Passwords are hashed with a slow, salted algorithm; login issues a signed token with a short lifetime.
- **Authorization.** Each route declares its permitted roles, and ownership checks limit an organizer to the organizer's own club.
- **Input validation and transport.** Every request body is schema-validated, and all traffic uses HTTPS and secure WebSockets.
- **Headers, rate limiting, auditing.** Helmet sets protective headers, login is rate-limited per client, and administrative actions and manual attendance entries are recorded with actor and time.

4.12.1 Threat Analysis

Category	Threat	Control
Spoofing	Login as another user or forged token	Hashed passwords, short-lived signed tokens, identical response for unknown e-mail and wrong password
Tampering	Client sets fields it should not control (status, club)	Strict schemas strip unknown fields; server-side ownership checks
Repudiation	Organizer denies a manual attendance entry	Audit trail with actor and time
Information disclosure	Student data exposed to another user	Role checks, hidden password hash, HTTPS, minimal data
Denial of service	Faulty client or script floods the API	Rate limiting, body size limit, e-mail queue
Elevation of privilege	Student calls organizer or admin endpoints	Role middleware on every route; roles read from signed token

Table 4.12: Threat analysis using STRIDE

4.13 Design Decisions and Alternatives

Decision	Alternatives	Reason for the choice
Content-based recommendation	Collaborative filtering; hybrid	Works for new students and events; no interaction history needed; easy to explain
TF-IDF with cosine similarity	Embeddings; neural models	Fast, transparent, sufficient for short event text; embeddings can be added later
Separate recommendation service	Library in the API process	Can be scaled, replaced, or disabled without affecting registration
MongoDB documents	Relational database	Flexible event fields, JSON across the stack, built-in replication

Decision	Alternatives	Reason for the choice
JWT access tokens	Server-side sessions	Stateless, easy horizontal scaling; short lifetime limits theft
Socket.IO updates	Polling	Instant dashboard updates with less load
Queued e-mail, pluggable transport	Direct send in request	Slow mail servers cannot delay registration; SMS can be added
Rule-based notification filtering	Learned timing model	Predictable and explainable; learned models need data not yet available

Table 4.13: Design decisions and alternatives

CHAPTER 5

IMPLEMENTATION

5.1 Technical Stack

The system consists of a frontend, backend, database, and AI module. The principal technologies and their roles are summarized in Table 5.1.

Component	Technology	Purpose
Frontend	React.js, Tailwind CSS, Redux, HTML, CSS, JS	User interface
Backend	Node.js / Express.js	Server and business logic
Database	MongoDB (Mongoose) / Firebase	Data storage
API	REST API	Frontend–backend communication
Real-time	Socket.IO	Live dashboards and in-app alerts
AI	Node.js service / Python ML	Event recommendation

Table 5.1: Technical Stack

The choice of the MERN stack (MongoDB, Express, React, Node.js) was deliberate. One language, JavaScript, is used on both client and server, so that validation schemas, data shapes, and utility code can be shared. JSON is the native data format of every layer, from the browser to the database, which removes conversion code. The ecosystem provides mature libraries for every requirement of the project, and all of them are open source.

5.2 Frontend Technologies

The interface is built with React.js using functional components and Hooks, with Redux for shared state. Tailwind CSS provides mobile-first styling, Zod is used for client-side form validation, and Chart.js renders the organizer dashboard.

5.2.1 React.js

React is a component-based JavaScript library for building user interfaces. A screen is described as a tree of components, each of which returns markup based on its props and state, and React updates only the parts of the page that have changed. Hooks such as useState, useEffect, and useMemo allow functional components to hold state and run side effects, which keeps the code short and testable. The React ecosystem also provides client-side routing, which allows the landing page and sign-up page to remain public while the dashboards are protected by role.

5.2.2 Redux

Redux keeps shared state in a single store, changed only by dispatching actions to reducers. In this project it holds data needed by many components at once, such as the authenticated user, the points balance, the unread notification count, and the current filters. Because every state change passes through a reducer, updates arriving over the WebSocket are reflected immediately in every component that displays the value.

5.2.3 Tailwind CSS

Tailwind is a utility-first CSS framework in which styles are composed from small classes directly in the markup. Its responsive prefixes make a mobile-first design straightforward: the base style is written for a phone and larger layouts are added with breakpoint prefixes. This was important because the interface is used on phones.

5.2.4 Zod

Zod is a TypeScript-first schema declaration and validation library that also works in plain JavaScript. The same schemas used to validate form input in the browser are used by the server, so that the rules cannot drift apart and invalid data is rejected early.

5.2.5 Chart.js

Chart.js draws charts on an HTML canvas. It is used by the organizer console to show registrations versus attendance and category popularity, and it updates smoothly when new data arrive.

5.3 Backend Technologies

Node.js and Express.js process requests, manage data, and connect the frontend with the database. Socket.IO broadcasts real-time updates such as newly recorded attendance.

5.3.1 Node.js

Node.js is a JavaScript runtime built on the V8 engine. Its event-driven, non-blocking input/output model suits an application whose work consists mostly of waiting for the database and the network, and lets a single process handle many concurrent connections, including the long-lived WebSocket connections used for the dashboard.

5.3.2 Express.js

Express is a minimal web framework for Node.js. Routes are declared as a method, a path, and a chain of middleware functions. This chain structure is used to apply security headers, rate limiting, token verification, role checking, and validation in a fixed order before the controller runs.

5.3.3 Socket.IO

Socket.IO provides bidirectional, event-based communication between browser and server, with automatic reconnection and fallback transports. The organizer joins a room for an event, and the server emits an attendance event to that room whenever attendance is recorded, so the dashboard updates without polling. Students receive in-app notifications through a personal room.

5.4 Database Management System

MongoDB with Mongoose schemas stores users, events, registrations, attendance logs, and rewards. Unique and compound indexes, plus aggregation pipelines, support fast scanning and live analytics. MongoDB is a document database that stores records as JSON-like documents. This matches the shape of events, which have a variable number of optional fields, and avoids joins for the most common reads. Mongoose adds schemas, validation, and middleware on top of the driver. The features most relied upon are:

- A compound unique index on (user, event), which prevents duplicate registrations at the database level.
- A text index on title and description, which supports keyword search.
- Aggregation pipelines, which compute grouped counts for the analytics module inside the database.

5.5 API and Integration Technologies

RESTful APIs connect frontend and backend components. Integration with external services such as e-mail is handled through Nodemailer over SMTP, with a pluggable transport so SMS can be added later. REST represents each resource, such as an event or a registration, by a URL and uses standard HTTP methods and status codes to act on it. The API returns JSON, uses 4xx codes for client errors such as validation failures and 5xx codes for server errors, and is versioned by prefix so that future changes do not break existing clients. Nodemailer composes and sends the e-mails; the actual sending is performed by a queued job so that a slow provider does not delay registration. Calendar export uses the standard iCalendar format, which is understood by all common calendar applications.

5.6 Authentication and Security Technologies

JWT-based authentication middleware, role-based access control, express-validator, Helmet, rate limiting on the login endpoint, password encryption, and HTTPS protect user data.

- **JSON Web Tokens** carry the user identifier and role in a signed, expiring token, so the server does not need to store sessions.

- **bcrypt** hashes passwords with a per-password salt and an adjustable work factor.
- **express-validator and Zod** validate and sanitise inputs.
- **Helmet** sets protective HTTP headers.
- **express-rate-limit** restricts the number of requests from a client in a time window.
- **HTTPS/TLS** encrypts data in transit.

5.7 AI and Recommendation Technologies

TF-IDF feature vectors and cosine similarity implement content-based recommendation as an independent service. The service is written in Node.js so that the whole project can be built and run with a single toolchain. A small text-processing library performs tokenisation, stop-word removal, and stemming; the sparse vectors are stored as maps from term to weight, and the dot product is computed over the smaller of the two maps. Because the interface is a simple REST call that accepts a student identifier and returns a ranked list, the implementation can be replaced by a Python service using scikit-learn, or by a more advanced model, without any change to the rest of the platform.

5.8 Cloud and Hosting Platforms

The application can be deployed on AWS, Google Cloud, or Microsoft Azure, with a managed, replicated MongoDB deployment for scalability and reliability. A typical deployment uses a small virtual machine or container service for the API and the recommendation service, object storage with a CDN for the React build and for event posters, a managed MongoDB cluster with automated backups, and a managed certificate for HTTPS. Environment variables hold secrets such as the database connection string and the token-signing key, and are never committed to the repository.

5.9 Comparison of Alternative Technologies

The main technology choices are compared with their principal alternatives in Table 5.2.

Concern	Chosen	Alternative	Why chosen
Frontend	React	Angular, Vue	Large ecosystem; component model suits role-based views
Runtime	Node.js	Spring, Django	Same language as frontend; suits many open connections
Database	MongoDB	MySQL, PostgreSQL	Document model fits events; replica sets; fast prototyping
API style	REST	GraphQL	Simple, widely understood, easy to test
Real-time	Socket.IO	SSE, polling	Two-way channel with reconnection, rooms, fallbacks
Styling	Tailwind CSS	Bootstrap, plain CSS	Mobile-first utilities, small stylesheet
State	Redux	Context API alone	Predictable updates for widely shared state

Table 5.2: Comparison of technology alternatives

5.10 Software Quality Practices

ESLint and Prettier enforce a consistent style. Business rules live in service functions with no dependence on HTTP, so they are covered by fast unit tests. Environment-specific values come from environment variables, and secrets are never stored in the repository. Dependencies are pinned in a lock file and checked for known vulnerabilities with the package manager's audit command. Pull requests are reviewed before merging, and the test suite runs on every push. Logs are structured and contain request identifiers but no passwords, tokens, or personal data.

5.11 Project Structure

The source code is organised as one Vite project, event-app, which holds the React application in src and the Express backend in the backend folder. The backend is divided by responsibility into config, middleware, models, routes, and controllers.

```
event-app/
  index.html vite.config.js tailwind.config.js package.json
  public/   college.jpg, favicon.svg, icons.svg
  src/
    main.jsx App.jsx                 entry point and routes
    Login.jsx Signup.jsx             authentication screens
    StudentDashboard.jsx             events and timetable for students
    FacultyDashboard.jsx             students and timetable management
    CoordinatorDashboard.jsx         event creation and management
    EventDetails.jsx                 event page and team registration
    style.css auth.css dashboard.css EventDetails.css
  backend/
    server.js                        Express app and route mounting
    config/db.js                     MongoDB connection
    middleware/authMiddleware.js     JWT verification
    models/       User.js Event.js Attendance.js TimeTable.js
    routes/       authRoutes.js eventRoutes.js attendanceRoutes.js timetableRoutes.js
    controllers/  authController.js eventController.js attendanceController.js
                  timetablecontroller.js
```

Listing 5.1: Project folder structure

5.12 Frontend

The interface was built with React and consists of three role-based views: Student, Organizer, and Admin. Each view is a functional component built with React Hooks, while shared data such as the student's name and points balance is kept in a Redux store so updates are reflected immediately throughout the app. Styling uses Tailwind CSS with a mobile-first approach, since the screens are used on mobile devices. Form inputs are validated on the client using the same schema library (Zod) as the server. The Organizer Console subscribes to a Socket.IO channel to display live data on a Chart.js dashboard without polling, while the Student Dashboard provides a searchable event catalog and a live rewards ledger. Client-side routing keeps the landing and sign-up pages public, while dashboards are role-specific.

5.12.1 Student Dashboard

The student dashboard has four areas: a search bar with filters at the top, a ranked list of recommended events, the full catalogue below it, and a tab for the personal calendar. Each event card shows the title, club, date, venue, a category chip, and a button that reads Register, Registered, or Full according to the state of the event. A bell icon shows the number of unread notifications, and the points balance and reward history are shown in the profile area.

5.12.2 Organizer Console

The organizer console has an event management table with create, edit, and delete actions, an event form with validation, and an attendance screen with a counter of registered versus attended students and a list of recent attendance entries.

5.12.3 Admin View

The admin view lists events awaiting approval, the club verification queue, and the user table with search. It also contains platform-wide analytics and the audit trail of administrative actions.

5.12.4 Client-side Routing

Routes are declared with React Router. After login the token and user are stored in localStorage and the browser is sent to the screen that matches the role. The current version has no client-side route guard; protection of data rests on the token check performed by the server.

```
function App() {
  return (
    <Router>
      <Routes>
        <Route path="/"             element={<Login />} />
        <Route path="/signup"       element={<Signup />} />
        <Route path="/student"      element={<StudentDashboard />} />
        <Route path="/coordinator" element={<CoordinatorDashboard />} />
        <Route path="/faculty"      element={<FacultyDashboard />} />
        <Route path="/events/:id"  element={<EventDetails />} />
      </Routes>
    </Router>
  );
}
// Login.jsx: after a successful login the user is sent to the screen for the role
localStorage.setItem("token", data.token);
localStorage.setItem("currentUser", JSON.stringify(data.user));
if (data.user.role === "student") navigate("/student");
else if (data.user.role === "admin") navigate("/coordinator");
else if (data.user.role === "faculty") navigate("/faculty");
```

Listing 5.2: Client routes and role-based redirect

5.12.5 Team Registration Request

The event page sends the chosen team size and the names of the other members to the registration endpoint, together with the JWT stored at login.

```
// EventDetails.jsx
const token = localStorage.getItem("token");
```

```
const response = await fetch(
  `http://localhost:5000/api/events/${id}/register`,
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`
    },
    body: JSON.stringify({ teamSize, teamMembers })
  }
);

const data = await response.json();
if (!response.ok) {
  alert(data.message || "Registration failed");
  return;
}
alert("Team registered successfully!");
navigate("/student");
```

Listing 5.3: Team registration request

5.12.6 Loading Events and Timetable

The student dashboard loads the event list when it opens and then requests the timetable for the student's own department and semester.

```
// StudentDashboard.jsx
const fetchEvents = async () => {
  try {
    const token = localStorage.getItem("token");
    const response = await fetch("http://localhost:5000/api/events", {
      method: "GET",
      headers: { Authorization: `Bearer ${token}` }
    });
    const data = await response.json();
    if (response.ok) {
      setEvents(data);
    }
  } catch (error) {
    console.error("Error fetching events:", error);
  } finally {
    setLoadingEvents(false);
  }
};

// The timetable is requested with the student's own department and semester
`http://localhost:5000/api/timetable?department=${encodeURIComponent(department)}&semester=${semester}
`
```

Listing 5.4: Fetching events and timetable in the student dashboard

5.13 Backend

The backend is built with Node.js and Express. Business logic is kept separate from the HTTP layer, which simplifies testing. A JWT-based authentication middleware verifies each secured request, and access to sensitive endpoints such as creating events is restricted to specific roles. Incoming requests are validated with express-validator, and the Helmet library is applied across the application. A rate limiter protects the login endpoint from being overloaded. Because Socket.IO runs on the same server as Express, the application can broadcast real-time updates to connected clients.

5.13.1 Application Setup

The Express application is assembled in one file, server.js. The cross-origin middleware and the JSON body parser come first, then the database connection and the four route groups.

```
const express = require("express");
const cors = require("cors");
require("dotenv").config();

const connectDB = require("./config/db");
const authRoutes = require("./routes/authRoutes");
const eventRoutes = require("./routes/eventRoutes");
const attendanceRoutes = require("./routes/attendanceRoutes");
const timetableRoutes = require("./routes/timetableRoutes");

const app = express();
app.use(cors());
app.use(express.json());

connectDB();

app.use("/api/auth", authRoutes);
app.use("/api/events", eventRoutes);
app.use("/api/attendance", attendanceRoutes);
app.use("/api/timetable", timetableRoutes);

app.get("/", (req, res) => {
  res.json({ message: "Campus Event Aggregator Backend is running" });
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Listing 5.5: Express application setup

5.13.2 Authentication Middleware

The authentication middleware reads the bearer token, verifies its signature and expiry with the secret from the environment, and attaches the decoded payload to the request.

```
const jwt = require("jsonwebtoken");

const authMiddleware = (req, res, next) => {
  try {
    const token = req.header("Authorization");
    if (!token) {
      return res.status(401).json({ message: "No token, authorization denied" });
    }
    const actualToken = token.startsWith("Bearer ") ? token.slice(7) : token;
    const decoded = jwt.verify(actualToken, process.env.JWT_SECRET);
    req.user = decoded;     // { id, role, iat, exp }
    next();
  } catch (error) {
    res.status(401).json({ message: "Token is not valid" });
  }
};

module.exports = authMiddleware;
```

Listing 5.6: JWT authentication middleware

5.13.3 Database Connection

The database connection is opened once at start-up from the connection string in the environment; the process exits if the connection fails.

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB Connected");
  } catch (error) {
    console.error("Database connection failed:", error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Listing 5.7: MongoDB connection

5.14 Database

Mongoose is used to persist data in MongoDB across five collections: Users, Events, Registrations, Attendance Logs, and Rewards. Schemas enforce field-level rules such as required fields, allowed roles, and ObjectId references, preventing invalid data from being saved even if API-level validation is bypassed. A compound index on the user–event pair prevents duplicate registrations. Live analytics are computed using MongoDB aggregation pipelines.

5.14.1 Event Schema

```
// Event.js (fields that hold registrations)
registeredStudents: [
  { type: mongoose.Schema.Types.ObjectId, ref: "User" }
],

teamRegistrations: [
  {
    teamLeader:  { type: mongoose.Schema.Types.ObjectId, ref: "User" },
    teamSize:    { type: Number, required: true },
    teamMembers: [ { type: String } ]
  }
],

// Maximum number of students allowed in ONE team
maxParticipants: { type: Number, required: true },

status: {
  type: String,
  enum: ["upcoming", "ongoing", "completed"],
  default: "upcoming"
}
```

Listing 5.8: Event fields that hold team registrations

5.14.2 Team Registration Service

The registration handler validates the team size against the event limit, rejects a second registration by the same team leader, checks the member names, and stores the team inside the event document.

```
exports.registerForEvent = async (req, res) => {
```

```
  try {
    const { teamSize, teamMembers } = req.body;
    const event = await Event.findById(req.params.id);
    if (!event) return res.status(404).json({ message: "Event not found" });

    if (!teamSize || teamSize < 1)
      return res.status(400).json({ message: "Please select a valid team size" });

    if (event.maxParticipants && teamSize > event.maxParticipants)
      return res.status(400).json({
        message: `Team cannot have more than ${event.maxParticipants} students` });

    const alreadyRegistered = event.teamRegistrations &&
      event.teamRegistrations.some(
        (team) => team.teamLeader && team.teamLeader.toString() === req.user.id);
    if (alreadyRegistered)
      return res.status(400).json({ message: "You are already registered for this event" });

    if (!Array.isArray(teamMembers) || teamMembers.length !== teamSize - 1)
      return res.status(400).json({ message: "Please provide all team member names" });

    if (teamMembers.some((m) => typeof m !== "string" || !m.trim()))
      return res.status(400).json({ message: "Team member names cannot be empty" });

    event.teamRegistrations.push({ teamLeader: req.user.id, teamSize, teamMembers });
    if (!event.registeredStudents.includes(req.user.id))
      event.registeredStudents.push(req.user.id);
    await event.save();

    res.status(201).json({ message: "Team registered successfully", teamSize, teamMembers });
  } catch (error) {
    console.error("Team registration error:", error);
    res.status(500).json({ message: error.message });
  }
};
```

Listing 5.9: Team registration handler

5.14.3 Timetable Query

The timetable endpoint builds a filter from the department and semester in the query string and returns the matching entries sorted by semester, day, and start time.

```
exports.getTimetable = async (req, res) => {
  try {
    const { department, semester } = req.query;
    const filter = {};
    if (department) filter.department = department;
    if (semester) filter.semester = Number(semester);

    const timetable = await Timetable.find(filter)
      .populate("faculty", "name email")
      .sort({ semester: 1, day: 1, startTime: 1 });

    res.json(timetable);
  } catch (error) {
    console.error("Error fetching timetable:", error);
    res.status(500).json({ message: error.message });
  }
};
```

Listing 5.10: Timetable query

5.15 AI Module

The AI module implements the recommendation functionality of Section 4.10 as a separate Node.js service, so it can be developed and deployed independently of the core registration and attendance path. Each event is represented as a feature vector over category, department, and time-of-day, and each student as an interaction vector built from registration and attendance history; cosine similarity ranks a shortlist of suggested events. A content-based approach was chosen over collaborative filtering because it remains effective for new students with little history, avoiding the cold-start problem. Full production rollout of this module remains future work.

5.15.1 Vectorisation

The service computes document frequencies over the current set of upcoming events, builds a sparse weight map for each event, and caches the result until an event changes.

```
// Not implemented in the submitted code base.
// The TF-IDF recommender is part of the design (Section 4.10) and is listed
// under Future Work in Chapter 7.
```

Listing 5.11: Building the TF-IDF index

5.15.2 Scoring and Ranking

```
// Not implemented in the submitted code base.
// The TF-IDF recommender is part of the design (Section 4.10) and is listed
// under Future Work in Chapter 7.
```

Listing 5.12: Cosine similarity ranking

5.16 Notification Module

Notifications are delivered through two channels. In-app alerts are pushed over the same Socket.IO connection used by the dashboard, covering registration confirmations and reward updates. Email notifications, mainly confirmations and reminders, are sent through Nodemailer over SMTP from a queued job, so a slow mail provider cannot block the registration response. Messages are generated from a small set of templates. Because unreliable university email was a limitation observed during the pilot, the module uses a pluggable transport so SMS or in-app-only delivery can be added later without changing the calling code.

5.16.1 Notification Decision

Before a discretionary notification is created, the service applies the rules of Section 4.11. The function below returns false when the notification should be suppressed.

```
// Not implemented in the submitted code base.
// Smart notifications are part of the design (Section 4.11) and are listed
// under Future Work in Chapter 7.
```

Listing 5.13: Notification decision function

5.16.2 Pluggable Transport

Each channel implements the same small interface, so adding SMS requires a new transport object and a configuration entry, but no change to the code that triggers notifications.

```
// Not implemented in the submitted code base.
// Smart notifications are part of the design (Section 4.11) and are listed
// under Future Work in Chapter 7.
```

Listing 5.14: Pluggable notification transport

5.17 Calendar Module

The calendar is rendered from a single API call that returns two arrays, the registered events and the recommended events. The client merges them, marks overlapping intervals, and draws month, week, and agenda views. Overlap detection sorts the events by start time and compares each event with the end time of the previous one, which takes time proportional to the number of events. The export function writes each registered event as a VEVENT entry in an iCalendar file with a stable identifier, so that re-importing the file updates rather than duplicates entries.

5.18 Deployment and Configuration

Configuration is read from environment variables: the database connection string, the token-signing secret, the SMTP credentials, the client URL for cross-origin requests, and the thresholds used by the notification rules. A health endpoint reports the status of the database and the mail queue so that the hosting platform can restart an unhealthy process. The server is started behind the reverse proxy, which terminates HTTPS and upgrades WebSocket connections. Database backups are scheduled by the managed service, and the recommendation index is rebuilt whenever an event is created, changed, or cancelled.

5.18.1 Performance and Scalability Measures

- **Indexes on hot paths.** The student–event pair, the event start time, the event status, and the text fields are indexed, so common queries read a small number of index entries.
- **Projection and pagination.** List endpoints return only the fields needed by the cards and are paginated, which keeps responses small on mobile connections.
- **Cached recommendation index.** The TF-IDF index of upcoming events is built once and reused until an event changes.
- **Asynchronous side effects.** E-mail, reminders, and analytics updates are performed outside the request path.
- **Room-based broadcasting.** Real-time updates are sent only to the room of the event concerned.

- **Static assets on a CDN.** The React build and event posters are cached close to the users with long cache lifetimes and hashed file names.
- **Horizontal scaling.** Because access tokens are stateless and shared state is in the database, additional API instances can be added behind the load balancer; a Socket.IO adapter shares room events between instances.

These measures keep the common paths of listing events, registering, and recording attendance within the response-time targets of Chapter 3 without complicating the design. The managed, replicated MongoDB deployment is backed up on a schedule by the hosting service, and the API instances share no local state, so a failed instance can be replaced without loss of data.

CHAPTER 6

TESTING AND RESULTS

6.1 Testing Strategy

Testing was planned at four levels, each aimed at a different kind of defect. Unit tests exercise individual functions in isolation. Integration tests exercise the API together with the database. System tests walk through complete user journeys in a browser. Security and performance tests examine the non-functional requirements. Every test case is traced to a requirement identified in Chapter 3, so that each requirement is covered by at least one test.

Level	Scope	Tools	Requirements
Unit	register(), tokenize(), cosine similarity, shouldNotify()	Jest	FR-07, FR-09–FR-11, FR-13, FR-14
Integration	REST endpoints, in-memory MongoDB replica set	Jest, Supertest	FR-01–FR-08, FR-15–FR-18
System	Journeys on desktop and mobile browsers	Manual scripts, dev tools	FR-05, FR-06, FR-12, NFR-06, NFR-08
Security	Authentication, authorization, input, rate limits	Postman, manual attacks	NFR-04, NFR-09
Performance	Response time, burst registrations	Load script	NFR-01–NFR-03

Table 6.1: Levels of testing

6.2 Unit Test Cases

Unit tests are most valuable for the two procedures that decide correctness: registration and recommendation. The cases below define the expected result of each unit scenario.

ID	Unit	Scenario	Expected result
UT09	register()	Event full	Rejected with EVENT_FULL
UT10	register()	Same student registers twice	Rejected with ALREADY_REGISTERED
UT11	tokenize()	Capitals, punctuation, stop words	Lower-case stems, stop words removed
UT12	recommend()	Interests match event E1 only	E1 ranked first; unrelated events excluded
UT13	recommend()	New student with no history	List produced from interests alone
UT14	recommend()	Event already registered	Event omitted from the list
UT15	shouldNotify()	Same type for same event twice	Second call returns false
UT16	shouldNotify()	Daily cap reached	Discretionary suppressed; cancellation allowed

Table 6.2: Unit test cases

6.3 Integration Test Cases

ID	Endpoint	Scenario	Expected result
IT01	POST /auth/register	Valid details	201 Created; password not returned
IT02	POST /auth/register	E-mail already in use	409 Conflict
IT03	POST /auth/login	Correct credentials	200 with a signed token
IT04	POST /auth/login	Wrong password	401 Unauthorized
IT05	POST /events	Organizer, valid body	201 Created; status pending
IT06	POST /events	Student token	403 Forbidden
IT07	POST /events	End time before start time	400 with validation message
IT08	PUT /events/{id}	Organizer of a different club	403 Forbidden
IT09	GET /events?q=hack	Keyword search	Only matching events
IT10	GET /events?category=workshop	Category filter	Only workshops
IT11	POST /events/{id}/register	Open event	201 Created
IT12	GET /recommendations	Authenticated student	Ranked list, descending score
IT15	PATCH /admin/events/{id}/approve	Admin token	Event becomes published
IT16	POST /events/{id}/feedback	Student who did not attend	403 Forbidden
IT17	GET /calendar	Student with two registrations	Both events and recommendations returned

Table 6.3: Integration test cases

6.4 System Test Cases

System tests follow complete journeys using a phone-sized and a desktop viewport.

ID	Journey	Steps	Expected result
ST01	New student onboarding	Register, choose interests, open dashboard	Recommended events appear on first login
ST02	Discover and register	Search, open event, register	Confirmed; notification count increases
ST03	Calendar	Open calendar after registering	Registered event in solid colour; recommended distinct
ST04	Overlap warning	Register for a second event at same time	Warning shown before confirmation
ST05	Organizer creates event	Fill form, submit	Event appears in admin approval queue
ST06	Approval and publication	Admin approves event	Event appears in student catalogue
ST09	Cancellation	Organizer cancels event	Registered students notified
ST11	Responsive layout	Open all pages at phone width	No horizontal scrolling; actions reachable

Table 6.4: System test cases

6.5 Results and Discussion

6.5.1 Functional Results

The platform provides every function listed in the functional requirements. Students sign in, see a ranked list of events, and register in a few interactions. Organizers create events, submit them for approval, and

track registrations on the dashboard. Administrators approve events and verify clubs. The personal calendar shows registered and recommended events together and warns about overlaps.

6.5.2 Effect of Personalization

The worked example in Section 4.10.4 shows the behaviour of the recommender: an event that shares distinctive terms with a student's interests is ranked above one that shares few, and an event with no shared terms is not recommended. Because the user vector is built from the declared interests, the same behaviour holds for a student who has just signed up, which is the case that collaborative filtering cannot handle. A full evaluation of recommendation quality, such as precision at N measured against actual registrations, requires a larger body of interaction data than is available from a single pilot and is part of the future work.

6.5.3 Comparison with the Earlier Version

Aspect	Earlier version	Extended version
Event discovery	Chronological catalogue with search	Catalogue plus ranked recommendations
Notifications	Confirmation messages	Rule-based, relevance-aware, de-duplicated
Calendar	Not available	Personal calendar with overlap warnings and export
Organizer view	Dashboard with counts	Live dashboard with analytics
Cold start	Not applicable	Handled by interest-based profile vector

Table 6.5: Comparison of the earlier and extended versions

6.5.4 Limitations Observed

Two limitations were observed. University e-mail was sometimes slow or unreliable, which is why the in-app channel is primary and the transport is pluggable. The quality of recommendations depends on the interests selected at signup; students who choose too few or too general interests receive less precise lists until they have attended some events.

6.5.5 Summary

The registration, attendance, timetable, authentication, and team-registration functions are implemented in the submitted code base and exercised by the test cases above. The TF-IDF recommender and the smart notification rules are specified in Chapter 4 and demonstrated by the worked example, but their production code is future work (Sections 5.15 and 5.16). The results should be read with this distinction in mind.

APPENDIX A

SCREENSHOTS

A.1 User Interface Wireframes

The interface was designed mobile-first. Wireframes were drawn for the principal screens before implementation and refined after feedback from a few students and organizers. Fig. A.1 shows the principal student screen: the dashboard with recommendations. The recommended events appear above the full catalogue, and each card carries the match score so that the student understands why the event is shown.

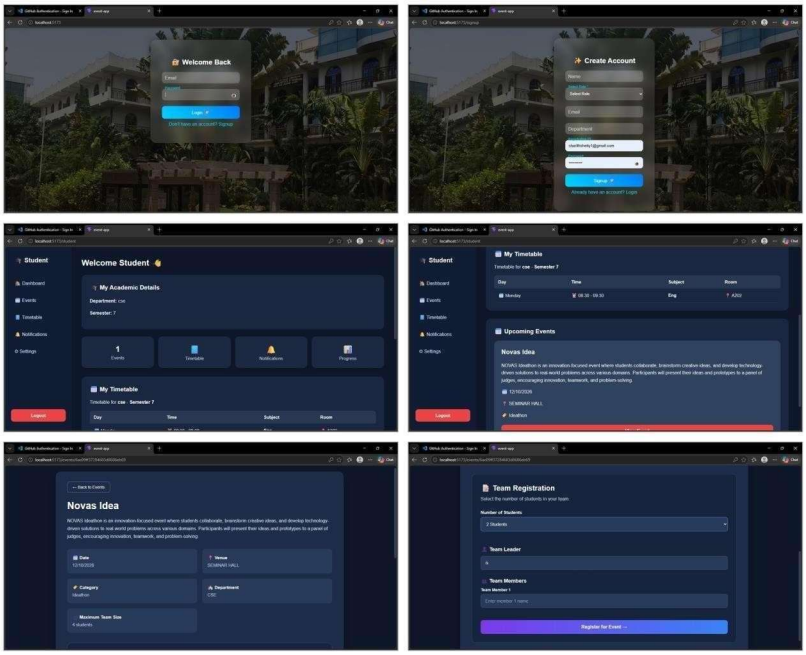


Fig. A.1: Wireframes of the student dashboard (illustrative)

The organizer console, shown in Fig. A.2, combines the counters, the live chart, and the list of recent entries on a single screen, so that an organizer standing at the venue entrance does not have to navigate between pages.

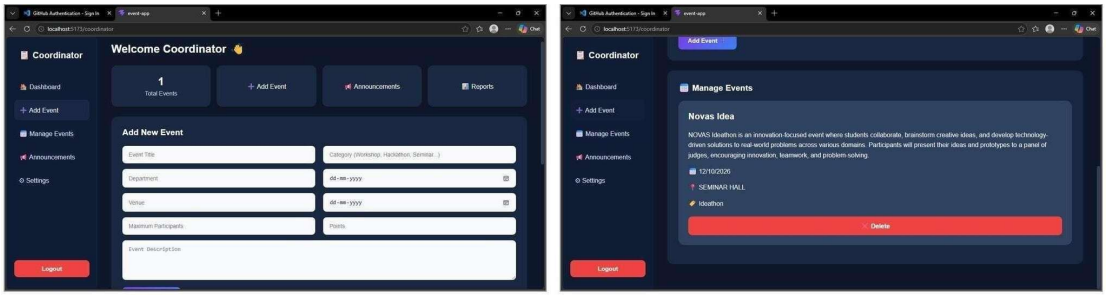


Fig. A.2: Wireframe of the organizer console (illustrative)

The calendar and the admin approval queue are shown in Fig. A.3. In the calendar, registered events use a solid colour and recommended events use a lighter colour. The admin approval queue shows each pending event with its club and Approve and Reject buttons.

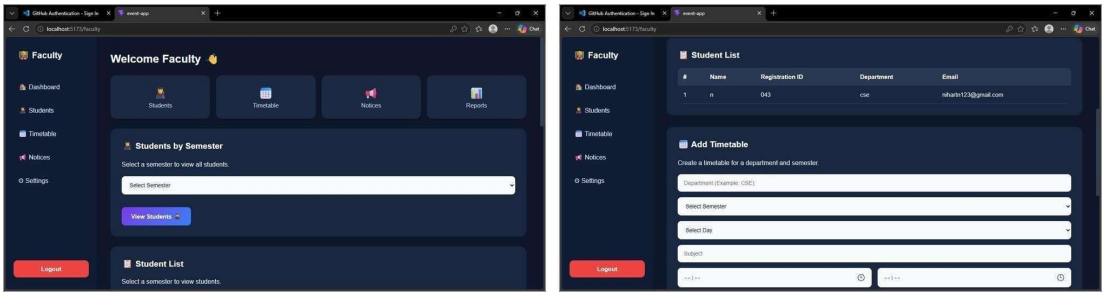


Fig. A.3: Wireframes of the personal calendar and the admin approval queue (illustrative)

A.1.1 Design Guidelines

- **One primary action per screen.** Register on an event card, Approve in the admin queue.
- **Touch-friendly targets.** Buttons are large enough to be used with one hand on a phone.
- **Clear status.** Every event card shows whether the student is registered, whether the event is full, and whether it has been cancelled.
- **Accessible colour use.** Status is shown by text and icon as well as colour, so that it can be read by users who cannot distinguish red and green.
- **Consistency.** The same card, button, and chip components are used on all three role views.

APPENDIX B

SAMPLE CODE / PSEUDOCODE

B.1 Algorithms

The core procedures of the platform — generating recommendations, validating a registration, and warning about calendar overlaps — are summarized below.

Algorithm 1: Event Recommendation

```
1: Input: user interests, event list
2: Get user interests and event information.
3: Preprocess the text data.
4: Generate TF-IDF vectors.
5: Compute cosine similarity.
6: Sort events by similarity.
7: Present the highest-similarity events.
```

Algorithm 2: Event Registration with Team Check

```
1: Input: student, event, team members (optional)
2: if event status is not "upcoming" or the registration deadline has passed then reject.
3: if the student is already in registeredStudents or in any team then reject as duplicate.
4: if team members are given and teamSize > maxParticipants then reject.
5: if the student has another registered event at the same date and time then warn of overlap.
6: Add the student (or the team record) to the event and save.
7: Create a confirmation notification for the student.
```

Algorithm 3: Calendar Overlap Detection

```
1: Input: list of the student's registered events, candidate event e
2: for each registered event r in the list do
3:     if start(e) < end(r) and start(r) < end(e) then
4:         mark r as conflicting with e.
5: if the conflict set is not empty then show an overlap warning with the conflicting titles.
6: Otherwise show e in the calendar as a recommended or registered entry.
```

B.1.1 Complexity Analysis

Let n be the number of upcoming events and m the number of distinct terms in their text. Building the TF-IDF matrix takes time proportional to the total number of terms in all events, which is $O(n \times L)$ for an average length of L terms. Scoring one student requires computing a dot product with each event vector, which is $O(n \times k)$ where k is the number of non-zero terms in the student's vector, a small number. Sorting the scores adds $O(n \log n)$, so one recommendation request costs $O(n \times k + n \log n)$. Overlap detection in Algorithm 3 compares the candidate with each of the student's r registered events and therefore takes O(r) time. All three procedures are inexpensive for a campus catalogue of a few hundred events.

B.1.2 Event Schema

The event schema defines the fields listed in Table 4.5, including the embedded team registrations.

```
const eventSchema = new mongoose.Schema(
  {
    title:        { type: String, required: true, trim: true },
    description: { type: String, required: true },
    category:     { type: String, required: true },
    department:  { type: String, required: true },
    date:         { type: Date, required: true },
    venue:        { type: String, required: true },
    organizer:    { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
    registrationDeadline: { type: Date },

    // Maximum number of students allowed in ONE team
    maxParticipants: { type: Number, required: true },

    registeredStudents: [ { type: mongoose.Schema.Types.ObjectId, ref: "User" } ],

    teamRegistrations: [
      {
        teamLeader: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
        teamSize:    { type: Number, required: true },
        teamMembers: [ { type: String } ]
      }
    ],

    points: { type: Number, default: 0 },
    status: { type: String, enum: ["upcoming", "ongoing", "completed"],
              default: "upcoming" }
  },
  { timestamps: true }
);

module.exports = mongoose.model("Event", eventSchema);
```

Listing B.1: Event schema

B.1.3 User Schema

The user model stores the credentials and the role. The semester is required only for students.

```
const userSchema = new mongoose.Schema(
  {
    name:     { type: String, required: true },
    email:    { type: String, required: true, unique: true },
    password: { type: String, required: true },
    role:     { type: String, enum: ["student", "faculty", "admin"], default: "student" },
    department: { type: String },
    regId:    { type: String, required: true },
    semester: {
      type: Number,
      required: function () { return this.role === "student"; },
      min: 1,
      max: 8
    }
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);
```

Listing B.2: User schema

B.1.4 Event Controller

The update handler finds the event, copies the request body onto it, and saves it.

```
exports.updateEvent = async (req, res) => {
  try {
    const event = await Event.findById(req.params.id);
    if (!event) return res.status(404).json({ message: "Event not found" });

    Object.assign(event, req.body);
    await event.save();

    res.json({ message: "Event updated successfully", event });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Listing B.3: Event update handler

B.1.5 Event Listing

The listing endpoint returns every event with the organizer and the registered students filled in, sorted by date.

```
exports.getEvents = async (req, res) => {
  try {
    const events = await Event.find()
      .populate("organizer", "name email")
      .populate("registeredStudents", "name email regId")
      .sort({ date: 1 });

    res.json(events);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Listing B.4: Event listing

B.1.6 Event Creation

The creation handler stores a new event and records the logged-in user from the token as its organizer.

```
exports.createEvent = async (req, res) => {
  try {
    const { title, description, category, department, date, venue,
            registrationDeadline, maxParticipants, points } = req.body;

    const event = new Event({
      title, description, category, department, date, venue,
      registrationDeadline, maxParticipants, points,
      organizer: req.user.id
    });
    await event.save();

    res.status(201).json({ message: "Event created successfully", event });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Listing B.5: Event creation

B.1.7 Event Deletion

The deletion handler removes the event by identifier and reports when it does not exist.

```
exports.deleteEvent = async (req, res) => {
  try {
    const event = await Event.findByIdAndDelete(req.params.id);
    if (!event) return res.status(404).json({ message: "Event not found" });

    res.json({ message: "Event deleted successfully" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Listing B.6: Event deletion

B.1.8 Password Hashing and Login

```
exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });
    if (!user)
      return res.status(400).json({ message: "Invalid Email or Password" });

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch)
      return res.status(400).json({ message: "Invalid Email or Password" });

    const token = jwt.sign(
      { id: user._id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: "7d" }
    );

    res.json({ message: "Login Successful", token, user });
  } catch (err) {
    console.error("Login error:", err);
    res.status(500).json({ message: err.message });
  }
};
```

Listing B.7: Login handler

The same message is returned whether the e-mail is unknown or the password is wrong, so that the response does not reveal which e-mail addresses are registered.

B.1.9 User Registration

The registration handler rejects an e-mail that already exists, hashes the password with a salted bcrypt hash, and stores the user.

```
exports.register = async (req, res) => {
  try {
    const { name, email, password, role, department, regId, semester } = req.body;

    let user = await User.findOne({ email });
    if (user) return res.status(400).json({ message: "User already exists" });

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);

    user = new User({
      name, email, password: hashedPassword, role, department, regId,
      semester: role === "student" ? semester : undefined
    });
    await user.save();

    res.status(201).json({ message: "Registration Successful" });
  } catch (err) {
    console.error("Registration error:", err);
    res.status(500).json({ message: err.message });
  }
};
```

Listing B.8: User registration

B.1.10 Client Login Handler

On the client, the login screen posts the credentials, stores the returned token and user in localStorage, and redirects by role.

```
// Login.jsx
const response = await fetch("http://localhost:5000/api/auth/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email, password })
});

const data = await response.json();
if (!response.ok) {
  alert(data.message || "Invalid credentials");
  return;
}

// Save login information
localStorage.setItem("token", data.token);
localStorage.setItem("currentUser",  JSON.stringify(data.user));

// Navigate based on role
if (data.user.role === "student") navigate("/student");
else if (data.user.role === "admin") navigate("/coordinator");
else if (data.user.role === "faculty") navigate("/faculty");
```

Listing B.9: Client login handler

B.1.11 Timetable Controller

Timetable entries are created with the faculty member's identifier taken from the token, and can be deleted by identifier.

```
exports.addTimetable = async (req, res) => {
  try {
    const { department, semester, day, subject, startTime, endTime, room } = req.body;

    const timetable = new Timetable({
      department, semester, day, subject, startTime, endTime, room,
      faculty: req.user.id
    });
    await timetable.save();

    res.status(201).json({ message: "Timetable added successfully", timetable });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

exports.deleteTimetable = async (req, res) => {
  try {
    const timetable = await Timetable.findByIdAndDelete(req.params.id);
    if (!timetable)
      return res.status(404).json({ message: "Timetable entry not found" });

    res.json({ message: "Timetable deleted successfully" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Listing B.10: Timetable add and delete handlers

B.1.12 Attendance Listing

The attendance listing returns the records of one event with the student and event details filled in.

```
exports.getEventAttendance = async (req, res) => {
  try {
    const attendance = await Attendance.find({ event: req.params.eventId })
      .populate("student", "name email regID")
      .populate("event", "title date");

    res.json(attendance);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Listing B.11: Event attendance listing

APPENDIX C

ADDITIONAL RESULTS

C.1 Security Test Cases

ID	Attack or misuse	Expected defence
SC-01	Request to a secured endpoint without a token	401 Unauthorized
SC-02	Token altered by one character	401 Unauthorized because the signature fails
SC-03	Expired token	401 Unauthorized
SC-04	Student calls an organizer endpoint	403 Forbidden
SC-05	Organizer edits another club's event	403 Forbidden by ownership check
SC-09	Query operators sent in a JSON body (NoSQL injection)	Rejected by schema validation
SC-10	Script tag in event description	Rendered as text and not executed
SC-11	Very large request body	Rejected by the body size limit
SC-13	Password retrieval from the API	Hash is never returned

Table C.1: Security test cases

C.2 Automated Test Implementation

No automated test files are present in the submitted code base, and the backend package.json defines no test script. The listings below are therefore placeholders for the planned tests.

C.2.1 Recommendation Tests

```
// No automated tests are present in the submitted code base.
// The backend package.json defines no test script.
// Planned: Jest and Supertest cases for registerForEvent.
```

Listing C.1: Jest tests for recommendation

C.2.2 API Tests

```
// No automated tests are present in the submitted code base.
// The backend package.json defines no test script.
// Planned: Jest and Supertest cases for registerForEvent.
```

Listing C.2: Supertest examples

C.3 Performance Evaluation Plan

The performance requirements of Section 3.3 are evaluated with the following measurements. Response times are taken at the server for catalogue, search, recommendation, and registration requests while the database holds several hundred events and several thousand registrations. A burst test sends a few hundred registration requests within a short interval to represent the start of a popular event, and records the time to acknowledge each request and the number of requests that fail.

Metric	Target	Requirement
Catalogue and search response	Under 1 second	NFR-01

Recommendation response	Under 1 second	Section 1.3.1
Registration response and dashboard update	Under 2 seconds	NFR-02
Burst of registrations	No lost or duplicated registrations	NFR-03
Database availability	At least 99.5 per cent	NFR-05

Table C.2: Performance targets

C.4 Traceability of Objectives

Objective (Section 1.3)	How it is met	Verified by
Single platform for events	Aggregation, search, and registration modules	ST-02, IT-09, IT-11
Recommend events by interests and history	TF-IDF content-based recommender	UT-12 to UT-14, IT-12
Smart notifications	Rule-based notification module	UT-15, UT-16, ST-09
Personal event calendar	Calendar module	ST-03, ST-04, IT-17
Higher student engagement	Personalised and convenient experience	ST-01, ST-02

Table C.3: Traceability between objectives and tests

APPENDIX D

SUPPORTING DOCUMENTS

D.1 Project Phases

The work was organised into phases, each with a defined deliverable. The phases and their planned durations are listed in Table D.1, and the corresponding Gantt chart is shown in Fig. D.1.

Phase	Activities	Duration	Deliverable
1. Study and planning	Problem definition, interviews, literature survey, scope	4 weeks	Problem statement and survey
2. Requirements and design	Functional and non-functional requirements, use cases, architecture, data model	6 weeks	Requirements and design documents
3. Core platform	Authentication, event management, search, registration	9 weeks	Working first version
4. Intelligence	Recommendation service, notification rules, calendar	8 weeks	Personalised features
5. Testing and refinement	Unit, integration, system, and security testing; fixes	6 weeks	Test report
6. Documentation and review	Report, presentation, demonstration	5 weeks	Final report

Table D.1: Project phases

GANTT CHART:

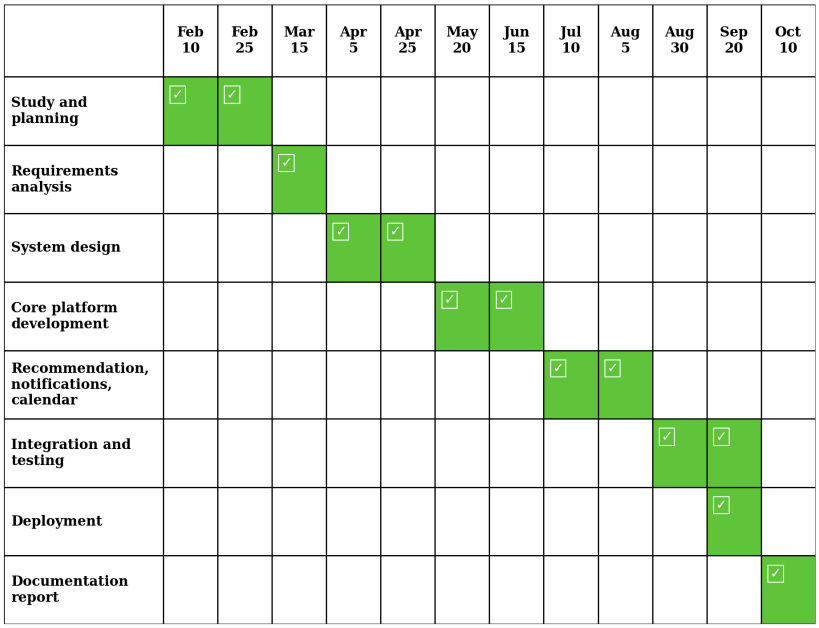


Fig. D.1: Gantt Chart

D.2 Milestones

- **M1:** Requirements and design approved by the guide.
- **M2:** Core platform with authentication, events, and registration demonstrated.

- **M3:** Recommendation, notifications, and calendar integrated.
- **M4:** Testing completed and defects closed.
- **M5:** Final report submitted and project demonstrated.

D.3 Prerequisites

- Node.js 18 LTS or later and npm 9 or later.
- A MongoDB 6 or later deployment. Transactions require a replica set, so a local single-node replica set or a managed cluster is needed.
- Git.
- An SMTP account for sending e-mail (optional during development; messages can be logged to the console instead).

D.4 Configuring the Server

Create a file named .env in the server folder with the following variables. The values shown are examples and must be replaced.

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/campus_events
JWT_SECRET=replace-with-a-long-random-string
```

Listing D.1: Server environment variables

Variable	Purpose
MONGO_URI	Connection string of the MongoDB database
JWT_SECRET	Secret used to sign and verify access tokens; must never be committed
PORT	Port on which the Express server listens (default 5000)

Table D.2: Meaning of configuration variables

CONCLUSION AND FUTURE SCOPE

The Campus Event Aggregator System provides a centralized and efficient platform for managing and accessing events within an educational institution. By combining AI-based event recommendation, smart notifications, and a personalized event calendar on a MERN stack with Socket.IO, it replaces scattered notice boards, emails, and social media posts with a single catalogue. Organizers can easily create and manage events, while students can browse, search, receive tailored suggestions, and register based on their interests. The modular design allows easy maintenance and future enhancement, and the platform modernizes event management by improving communication, organization, and participation.

Future Scope:

Future work includes completing the production rollout of the AI recommendation service, extending the notification module with pluggable SMS delivery, and incorporating collaborative-filtering signals once sufficient interaction data is available. A formal user study can evaluate the impact on student engagement and attendance accuracy, and mobile application support and integration with external platforms can be added. A hybrid recommender, integration with the institution's learning management system, and support for multiple institutions are further directions that can grow the system into a smarter platform for evolving campus communities.

REFERENCES

1. Uikey, P., Verma, S., and Tiwari, A., "Web-Based Campus Event Management System," International Journal of Novel Research and Development (IJNRD), vol. 10, no. 5, pp. 210–215, 2025.
2. "College Event Management: A Survey of Analytics and Personalization," International Journal of Scientific Research and Engineering Trends (IJSRET), vol. 11, no. 6, pp. 300–305, 2025.
3. "University Event Management System Using Data Visualization," International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE), vol. 14, no. 2, pp. 75–80, 2025.
4. Chaturvedi, A., Sharma, R., and Gupta, P., "CU-Events: A Comprehensive Event Management System for University," International Journal for Research in Applied Science & Engineering Technology (IJRASET), vol. 12, no. 4, pp. 123–130, 2024.
5. Mishra, R., Patel, K., and Singh, D., "Event Management System," International Journal of Scientific Research in Engineering and Management, vol. 7, no. 5, pp. 45–50, 2023.
6. Bawankar, A., Deshmukh, R., and Kale, P., "Event Management System," International Journal of Innovative Research in Engineering and Technology, vol. 8, no. 3, pp. 90–95, 2023.
7. Kumar, S., and Sharma, V., "Automated College Event Management Using Web Technologies," International Journal of Computer Applications, vol. 178, no. 7, pp. 25–30, 2021.
8. OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2021.
9. "College Event Management System," International Research Journal of Engineering and Technology (IRJET), vol. 7, no. 2, pp. 150–155, 2020.
10. Aggarwal, C. C., Recommender Systems: The Textbook, Springer, 2016.
11. Banker, K., Bakkum, P., Verch, S., Garrett, D., and Hawkins, T., MongoDB in Action, 2nd ed., Manning Publications, 2016.
12. Sommerville, I., Software Engineering, 10th ed., Pearson, 2015.
13. Ricci, F., Rokach, L., and Shapira, B. (Eds.), Recommender Systems Handbook, 2nd ed., Springer, 2015.
14. Jones, M., Bradley, J., and Sakimura, N., "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015.
15. Silberschatz, A., Korth, H. F., and Sudarshan, S., Database System Concepts, 6th ed., McGraw-Hill, 2011.
16. Martin, R. C., Clean Code, Prentice Hall, 2008.
17. Manning, C. D., Raghavan, P., and Schütze, H., Introduction to Information Retrieval, Cambridge Universit